Bộ câu trả lời phỏng vấn lập trình viên toàn diện MES

Tài liệu gồm **103 câu hỏi**, chia thành 10 nhóm, theo đúng thứ tự đề bài. Dấu ⭐ giữ nguyên để ưu tiên ôn tập. Nội dung giải thích bằng tiếng Việt; tên công nghệ, định danh và mã nguồn giữ cách viết chuyên môn.

Cách sử dụng

- Với câu hỏi kỹ thuật: trình bày nguyên lý, giải thích quyết định và đưa ví dụ; dành phần chi tiết cho câu hỏi tiếp theo của người phỏng vấn.
- Với câu hỏi kinh nghiệm: các đoạn "Câu trả lời theo CV" đã được cá nhân hóa cho Đào Mạnh Quân. Chỗ còn ngoặc vuông là thông tin CV chưa cung cấp, như nguyên nhân nghỉ việc hoặc một sự cố cụ thể; không tự điền bằng câu chuyện giả định.
- Phân biệt ba loại nội dung: "Theo CV" là trải nghiệm bạn khai trong hồ sơ; "Đối chiếu mã nguồn" là điều đã quan sát trong repo; "Ví dụ/đề xuất thiết kế" là cách giải bài toán, chưa tự đồng nghĩa bạn đã triển khai.
- Các con số, mã lệnh và mã lô trong **ví dụ kỹ thuật** là giả định, không phải số liệu thành tích. Số năm kinh nghiệm, mốc công việc và quy mô nhóm được nêu theo CV ở phần hồ sơ và câu trả lời cá nhân.
- Đã đối chiếu CV cá nhân 4 trang: khoảng 5 năm kinh nghiệm .NET và SQL Server; các dự án TMSS, TYT, Booking API, OESMedicalAPI cùng kinh nghiệm tại Edsolab và Twenty-Tech. Thông tin nghề nghiệp dưới đây dựa trên CV do bạn cung cấp, không phải kết quả xác minh độc lập với công ty.
- Mã C#, SQL, TypeScript và cấu hình dưới đây phục vụ giải thích, có thể lược bỏ phần khai báo hoặc xử lý phụ. Không phải bộ mã đã chạy kiểm thử hoặc kịch bản triển khai database.

Căn cứ riêng cho TYT: source hiện tại có nhiều host theo domain trong TYT.AppHost, gRPC code-first, RabbitMQ, truy cập SQL qua repository/Dapper, Redis, cùng các thành phần Loyalty Inbox/Outbox. Điều này xác nhận thành phần tồn tại trong source, không xác nhận ai viết, độ tin cậy production hay tính đúng đắn của mọi luồng đồng thời.

Điểm kiến trúc cần nói chính xác: hệ thống TYT được tổ chức theo module/layer, nhưng nhiều Domain Manager chạy thành ứng dụng riêng. Mô tả phù hợp là dịch vụ theo domain, kích thước tương đối lớn, có đặc điểm microservices. Không nên mô tả toàn hệ thống là modular monolith thuần túy chỉ vì dùng chung solution.

Hồ sơ dùng để cá nhân hóa câu trả lời

Nguồn: CV cá nhân do bạn cung cấp, 4 trang. Chỉ sử dụng thông tin nghề nghiệp cần cho phỏng vấn; không đưa số điện thoại, email, ngày sinh hoặc địa chỉ vào bộ câu trả lời.

Nội dung	Dữ kiện trong CV	Cách sử dụng khi phỏng vấn
Trọng tâm chuyên môn	.NET và SQL Server khoảng 5 năm; RabbitMQ/Redis khoảng 3 năm	Giới thiệu nền tảng backend vững; không cộng dồn các công việc song song thành thâm niên
Giao diện	Angular khoảng 2 năm, React khoảng 1 năm	Nêu có kinh nghiệm toàn diện, thế mạnh chính vẫn là backend
Nexus Frontier Tech	02/2025–09/2026	Gọi là công việc gần đây nhất theo mốc công ty trong CV
TMSS	Hệ thống hậu mãi ô tô đa tenant; nhóm 8 người	Ví dụ quản lý khách hàng/xe, báo giá, quyết toán, phụ tùng, phân quyền và đồng bộ
TYT	Hệ sinh thái Toyota/Lexus; nhóm 12 người	Ví dụ Loyalty, điểm, voucher, đăng nhập hợp nhất, gRPC và sự kiện
Booking API	09/2024–05/2025; vai trò backend; nhóm 6 người	Liên hệ giữ chỗ, hết hạn và cạnh tranh tài nguyên
OESMedicalAPI	01/2023–09/2024; vai trò backend; nhóm 6 người	Liên hệ nhiều tenant, truy vấn sản phẩm động và lịch sử cập nhật

Nội dung	Dữ kiện trong CV	Cách sử dụng khi phỏng vấn
Twenty-Tech	09/2022–02/2025; C#/Python, thu thập/làm sạch dữ liệu, triển khai và hỗ trợ	Ví dụ dữ liệu đầu vào không sạch, vận hành và trao đổi khách hàng
Edsolab	03/2021–09/2022; báo cáo điện tử, họp thông minh, phản ánh kiến nghị, quản lý nội bộ	Ví dụ trạng thái quy trình, lịch lặp, thông báo và phối hợp khách hàng
Vicogroup	09/2020–11/2020; C#/.NET Framework MVC	Có thể nhắc ngắn khi hỏi quá trình bắt đầu nghề
Định hướng	Ngắn hạn trở thành lập trình viên cấp cao và thành viên nòng cốt; dài hạn kiến trúc sư giải pháp	Gắn lộ trình ba năm với năng lực và nghiệp vụ MES
MES/PLC	CV không nêu dự án triển khai MES hoặc lập trình PLC trực tiếp	Không tự biến kinh nghiệm TMSS/DMS thành kinh nghiệm MES/PLC

Các điểm cần diễn đạt thống nhất:

1. Mốc làm tại Nexus kết thúc 09/2026, trong khi TMSS/TYT còn ghi “hiện tại”; tài liệu dùng “gần đây nhất” và không tự khẳng định bạn vẫn đang làm ở Nexus.
2. TMSS ghi bắt đầu 01/2025 nhưng mốc Nexus là 02/2025. Nếu được hỏi, cần giải thích bằng lịch tham gia thật; không tự đoán là thử việc, làm trước hoặc chuyển công ty.
3. Freelancer 09/2022–05/2025 là công việc thứ hai, trùng với công việc chính. Nói rõ làm song song và không cộng thời gian trùng.
4. CV ghi TYT theo kiểu modular monolith và hơn 200 module. Mã nguồn đã đối chiếu có nhiều project và nhiều host riêng; số project không phải số module nghiệp vụ. Câu 8.1 dùng mô tả chính xác hơn.
5. CV có diễn đạt bảo đảm tuyệt đối không mất thông điệp. Câu trả lời nên nêu các cơ chế cụ thể và phạm vi kiểm chứng, tránh bảo đảm tuyệt đối chỉ dựa vào Inbox/Outbox.
6. Lý do kết thúc công việc, một sự cố vận hành cụ thể, bất đồng trong nhóm và khả năng đi công tác chưa được xác định trong CV. Những phần này vẫn cần trải nghiệm hoặc cam kết thật.

Mục lục

1. Giới thiệu bản thân và kinh nghiệm — 10 câu
2. C# / .NET / ASP.NET Core — 12 câu
3. SQL Server / EF Core / giao dịch — 14 câu
4. RabbitMQ / hệ phân tán / đồng thời — 12 câu
5. MES / ERP / nghiệp vụ sản xuất — 15 câu
6. PLC / máy quét / mã vạch / tích hợp — 8 câu
7. Giao diện Angular / React / TypeScript — 8 câu
8. Kiến trúc và thiết kế hệ thống — 8 câu
9. Redis / Docker / triển khai / chẩn đoán — 8 câu
10. Hành vi và xử lý tình huống — 8 câu

---

1. Giới thiệu bản thân và kinh nghiệm

1.1. ⭐ Giới thiệu bản thân và kinh nghiệm làm việc

Câu trả lời theo CV, khoảng 60–90 giây:

“Tôi là Đào Mạnh Quân, một lập trình viên có khoảng 5 năm kinh nghiệm với .NET và SQL Server. Thế mạnh của tôi là phát triển backend cho các hệ thống doanh nghiệp, đồng thời tôi có kinh nghiệm làm giao diện bằng Angular và React.

Gần đây nhất, tôi làm tại Nexus Frontier Tech, tham gia TMSS và hệ sinh thái TYT cho Toyota/Lexus. Ở TMSS, tôi phát triển các phần quản lý khách hàng và xe, báo giá, quyết toán và quy trình phụ tùng, cùng các chức năng phân quyền, nhiều tenant và xử lý nền. Với TYT, tôi tham gia các luồng khách hàng thân thiết, tích lũy và đổi điểm, voucher và tích hợp giữa các hệ thống.

Về kỹ thuật, tôi đã làm với EF Core, Dapper, RabbitMQ, Redis và gRPC. Tôi quan tâm cách giữ dữ liệu nhất quán khi có yêu cầu đồng thời hoặc sự cố kết nối.

Tôi muốn ứng tuyển vị trí MES để áp dụng nền tảng đó vào các bài toán lệnh sản xuất, vật tư và truy xuất nguồn gốc, đồng thời học sâu hơn nghiệp vụ nhà máy."

Phiên bản ngắn khi được yêu cầu giới thiệu nhanh:

"Tôi là Quân, có khoảng 5 năm kinh nghiệm .NET và SQL Server, kèm Angular/React. Dự án gần đây nhất của tôi tại Nexus là TMSS và TYT cho Toyota, tập trung vào nghiệp vụ doanh nghiệp, tích hợp, phân quyền và Loyalty. Tôi muốn chuyển kinh nghiệm về giao dịch và đồng bộ dữ liệu sang lĩnh vực MES."

Nếu hỏi quá trình trước Nexus: tóm tắt Edsolab với hệ thống báo cáo/quy trình, Twenty-Tech với C#/Python và xử lý dữ liệu, rồi các dự án backend ngoài giờ. Không đọc lần lượt mọi dòng trong CV; chỉ mở rộng theo câu hỏi.

1.2. Mô tả dự án gần đây nhất tại Nexus Frontier Tech

Câu trả lời theo CV:

"Tại Nexus Frontier Tech, tôi tham gia hai hệ thống liên quan là TMSS và hệ sinh thái TYT.

TMSS là nền tảng quản lý dịch vụ hậu mãi cho Toyota Việt Nam và mạng lưới đại lý, bao gồm lịch hẹn, lệnh sửa chữa, phụ tùng, quyết toán, bảo hành và Loyalty. Nhóm dự án có 8 người; vai trò của tôi là phát triển .NET Core và Angular, với nền tảng ABP Framework/ASP.NET Zero, SQL Server, EF Core và Dapper.

TYT là hệ sinh thái kết nối ứng dụng khách hàng Toyota/Lexus, các cổng vận hành như PMS, RMS, Service CMS và các nền tảng doanh nghiệp. Nhóm có 12 người. Tôi tham gia phát triển dịch vụ và các nghiệp vụ điểm, voucher, đăng nhập hợp nhất cùng luồng tích hợp dùng gRPC và RabbitMQ.

Một luồng phù hợp để tôi trình bày sâu là đổi điểm lấy ưu đãi và đồng bộ nghiệp vụ liên quan, vì nó thể hiện quy tắc giao dịch, tính nhất quán dữ liệu và xử lý khi có lỗi."

Ví dụ phân biệt vai trò: TMSS thiên về nghiệp vụ đại lý và dịch vụ hậu mãi; TYT kết nối trải nghiệm khách hàng với các hệ thống liên quan. Có trao đổi dữ liệu không có nghĩa hai hệ thống là cùng một ứng dụng.

Khi hỏi kiến trúc: nêu TMSS theo nền tảng ABP/ASP.NET Zero đúng CV; với TYT, nói chia theo domain, nhiều host riêng, gRPC và sự kiện. Không dùng công nghệ .NET Aspire của TYT để mô tả mọi thành phần TMSS.

Mốc thời gian: CV chưa thống nhất giữa thời gian công ty và chữ "hiện tại" ở dự án, nên dùng "trong giai đoạn tại Nexus" cho tới khi bạn xác nhận lịch thật.

1.3. ⭐ Trách nhiệm chính của bạn trong dự án TMSS là gì?

Câu trả lời theo CV:

"Trong TMSS, tôi phát triển cả backend .NET Core và giao diện Angular. Công việc của tôi có bốn nhóm chính.

Thứ nhất là nghiệp vụ: quản lý khách hàng và xe, xử lý báo giá, quyết toán và các luồng phụ tùng.

Thứ hai là tích hợp: tham gia thiết kế đồng bộ sự kiện qua RabbitMQ, dùng Inbox/Outbox và cơ chế nhận việc từ database để hỗ trợ retry, hạn chế xử lý trùng khi chạy nhiều instance.

Thứ ba là xác thực và phân quyền: triển khai MasterSSO, phân quyền theo vai trò và kiểm soát dữ liệu giữa Host Toyota Việt Nam và tenant đại lý.

Thứ tư là tác vụ nền: nhắc lịch SMS/Zalo, theo dõi bảo hành, dọn dữ liệu thông điệp và xuất Excel bất đồng bộ."

Ví dụ để trình bày sâu: người dùng đại lý thao tác trên Angular; API xác nhận danh tính, quyền và phạm vi đại lý; tầng nghiệp vụ kiểm tra trạng thái; repository dùng EF Core/Dapper hoặc procedure cho dữ liệu; bước đồng bộ được tách theo quy tắc của luồng.

Phân biệt rõ: CV liệt kê phạm vi toàn TMSS có lịch hẹn, bảo hành và Loyalty, nhưng phần trách nhiệm của bạn cụ thể hơn. Không tự nhận đã viết toàn bộ module chỉ vì chúng có trong mô tả hệ thống.

Câu hỏi tiếp có thể gặp: tenant lấy từ đâu, tài khoản Host được nhìn dữ liệu nào, SQL/procedure nhận tenant ra sao, worker nhận việc bằng thao tác nào, và khi hai instance chạy cùng lúc thì chuyện gì xảy ra? Chuẩn bị một đường đi cụ thể bạn thực sự biết.

1.4. Bài toán kỹ thuật khó nhất bạn từng giải quyết là gì?

Chủ đề nên chọn theo CV: đồng bộ TMSS–Loyalty/TYT đáng tin cậy khi chạy nhiều instance. Đây là chủ đề đã được CV nêu, không cần tự tạo thêm một sự cố cụ thể.

Câu trả lời dựa trên nhiệm vụ trong CV:

"Một bài toán kỹ thuật khó của tôi là đồng bộ dữ liệu giữa các hệ thống mà vẫn giữ đúng nghiệp vụ khi có lỗi kết nối hoặc xử lý lại.

Khó khăn là một giao dịch database có thể đã thành công nhưng việc gửi sự kiện chưa thành công. Ngược lại, sự kiện có thể đã gửi nhưng worker chưa kịp lưu trạng thái. Khi chạy nhiều instance, còn phải tránh việc cùng nhận một bản ghi queue.

Trong phần công việc này, tôi tham gia triển khai RabbitMQ kết hợp Inbox/Outbox. Ý định gửi được lưu cùng dữ liệu nghiệp vụ, worker nhận việc qua database, phía nhận lưu thông điệp và kiểm soát xử lý lặp. Khi trình bày chi tiết, tôi sẽ chỉ rõ luồng và đoạn SQL mà mình trực tiếp phụ trách.

Điều tôi rút ra là không chỉ xử lý trường hợp thành công: phải xác định rõ điểm commit, điểm xác nhận và kết quả nếu tiến trình dừng ở giữa."

Ví dụ kỹ thuật giả định: broker nhận E1 nhưng worker dừng trước khi đánh dấu. Gửi lại E1 phải không tạo thêm lần đổi điểm.

Giới hạn cần giữ: CV mô tả đã làm các cơ chế này, nhưng không nêu số liệu, tên sự cố hoặc kết quả thử lỗi. Không tự nói đã giảm bao nhiêu phần trăm, đã kiểm tra mọi trường hợp hay không bao giờ mất dữ liệu.

Phương án khác từ CV: bài toán phát hành/đổi voucher đồng thời trong TYT, hoặc giữ chỗ có thời hạn trong Booking API. Chọn chủ đề bạn có thể giải thích bằng mã nguồn và quy tắc thật.

1.5. Vì sao bạn muốn rời công ty hiện tại?

CV cho biết: mốc công ty Nexus là 02/2025–09/2026; chưa nêu lý do kết thúc. Vì vậy không tự nói bị cắt dự án, hết hợp đồng, thiếu cơ hội hay có mâu thuẫn.

Phần có thể nói theo định hướng đã ghi trong CV:

"Giai đoạn tại Nexus giúp tôi có kinh nghiệm với hệ thống nghiệp vụ lớn, nhiều tenant và tích hợp giữa các nền tảng. Bước tiếp theo, tôi muốn tìm một môi trường phù hợp để gắn bó lâu dài, phát triển thành thành viên kỹ thuật nòng cốt và đi sâu hơn vào thiết kế giải pháp.

Vị trí MES phù hợp với hướng đó vì kết hợp .NET, dữ liệu, giao diện và các bài toán vận hành thực tế. Tôi muốn sử dụng kinh nghiệm hiện có để đóng góp, đồng thời xây dựng hiểu biết vững về sản xuất."

Để trả lời trực tiếp câu "vì sao nghỉ": bổ sung đúng một câu về nguyên nhân thật trước đoạn định hướng trên. Ví dụ "Hợp đồng/dự án của tôi kết thúc" chỉ dùng nếu đúng thực tế; không dùng như dữ kiện đã xác nhận.

Nếu thực tế vẫn còn làm tại Nexus: đổi cách nói cho đúng và cập nhật mốc CV. Nếu đã kết thúc tháng 09/2026, dùng "công ty gần đây nhất" thay cho "công ty hiện tại".

Định hướng tìm nơi gắn bó lâu dài phù hợp phần mục tiêu trong CV; nó không thay thế thông tin về nguyên nhân nghỉ việc mà CV chưa cung cấp.

1.6. Vì sao bạn quan tâm vị trí lập trình viên toàn diện MES?

Câu trả lời gắn với CV:

"Tôi quan tâm MES vì đây là một bước phát triển phù hợp từ các hệ thống doanh nghiệp mà tôi đã làm. Ở TMSS, tôi làm với quy trình đại lý, dữ liệu khách hàng/xe, phụ tùng và quyết toán. Ở TYT, tôi xử lý các luồng điểm, voucher và đồng bộ giữa các hệ thống.

Những kinh nghiệm đó giúp tôi quen với dữ liệu nhiều trạng thái, ràng buộc giao dịch, phân quyền và xử lý yêu cầu đồng thời. Trong MES, các vấn đề này xuất hiện ở lệnh sản xuất, cấp phát vật tư, ghi nhận kết quả và tích hợp ERP.

Tôi cũng có nền tảng .NET/SQL Server cùng Angular và React, nên có thể tham gia từ API tới màn hình vận hành. Tôi muốn bổ sung nghiệp vụ BOM, routing, WIP, chất lượng và truy xuất để phát triển lâu dài trong lĩnh vực này."

Ví dụ liên hệ có giới hạn: giữ chỗ dịch vụ trong Booking API giúp giải thích tư duy hạn tài nguyên và hết hạn; đổi voucher giúp giải thích giao dịch và chống lặp. Chúng chưa phải kinh nghiệm định mức sản xuất hoặc lập trình thiết bị.

Khi nói về công ty tuyển dụng: chỉ nhắc yêu cầu đã có trong mô tả công việc. Chưa xác nhận được ERP, loại nhà máy hay cách triển khai thì không tự khẳng định.

1.7. Bạn từng làm hệ thống sản xuất hoặc ERP chưa?

Điểm xuất phát từ CV: các dự án đã nêu là hệ thống doanh nghiệp, DMS/hậu mãi ô tô, Loyalty, đặt dịch vụ và sản phẩm kho. CV không ghi một dự án MES, triển khai ERP sản xuất hoặc lập trình PLC trực tiếp.

Câu trả lời phù hợp nếu đúng là bạn chưa làm MES trực tiếp:

"Tôi chưa trực tiếp triển khai một MES hoàn chỉnh. Kinh nghiệm chính của tôi là TMSS và TYT cho Toyota, trong đó có quy trình phụ tùng, giao dịch nghiệp vụ, dữ liệu nhiều tenant và đồng bộ hệ thống. Trước đó tôi làm OESMedicalAPI về sản phẩm kho và Booking API có cơ chế giữ chỗ theo thời hạn.

Tôi hiểu đây là những kinh nghiệm liên quan về kỹ thuật, không đồng nghĩa đã làm nghiệp vụ nhà máy. Khi chuyển sang MES, tôi cần học thêm BOM, routing, WIP, ghi nhận tiêu hao, quản lý chất lượng và cách tích hợp thiết bị.

Tôi có thể bắt đầu từ các phần phù hợp nền tảng của mình như API, dữ liệu, phân quyền và tích hợp, rồi học quy trình sản xuất qua BA và người vận hành."

Nếu có kinh nghiệm MES/ERP ngoài CV: bổ sung tên hệ thống, module và vai trò thật. Không kết luận bạn hoàn toàn chưa từng làm chỉ vì CV không ghi.

Ví dụ cần phân biệt: TMSS là nền tảng dịch vụ hậu mãi/DMS; có phụ tùng và thanh toán không tự làm nó thành MES hoặc một ERP sản xuất.

1.8. ⭐ Dự án nào trước đây liên quan nhất đến MES?

Câu trả lời theo CV:

"Tôi chọn TMSS là dự án gần MES nhất về quy trình doanh nghiệp và dữ liệu nghiệp vụ, vì có quản lý phụ tùng, báo giá, quyết toán, phân quyền đại lý và tích hợp. Nếu xét độ tin cậy giao dịch và xử lý đồng thời, TYT là ví dụ bổ sung tốt với nghiệp vụ điểm và voucher.

Tôi cũng có hai dự án nhỏ hơn liên quan: Booking API với giữ chỗ và tự hết hạn, và OESMedicalAPI với dữ liệu sản phẩm, nhiều tenant và lịch sử cập nhật. Những bài toán này giúp tôi tiếp cận quản lý vật tư và lịch sử MES, nhưng không thay thế kinh nghiệm vận hành nhà máy."

Dự án trong CV	Kinh nghiệm đã nêu	Liên hệ MES khi giải thích
TMSS	Phụ tùng, trạng thái nghiệp vụ, Host/đại lý, đồng bộ	Vật tư, quy trình, phân quyền nhiều đơn vị

Dự án trong CV	Kinh nghiệm đã nêu	Liên hệ MES khi giải thích
TYT	Điểm, voucher, gRPC, sự kiện	Giao dịch đồng thời và tích hợp
Booking API	Chọn/giữ dịch vụ, hết hạn	Giữ chỗ tài nguyên và giải phóng
OESMedicalAPI	Sản phẩm động, nhiều tenant, lịch sử cập nhật	Danh mục vật tư và audit
Báo cáo điện tử	Tạo/sửa, phân phối/chuyển báo cáo	Trạng thái phê duyệt và dữ liệu báo cáo

Ví dụ: giữ một suất dịch vụ trong Booking API có nét giống giữ vật tư cho lệnh; khi hết hạn hoặc hủy thì giải phóng phần đã giữ. Trong MES cần thêm lô, đơn vị, chất lượng và việc vật tư đã được dùng thật hay chưa.

1.9. Bạn phối hợp với BA, QA và khách hàng như thế nào?

Căn cứ CV: tại Edsolab và Twenty-Tech, bạn có công việc trao đổi thường xuyên với quản lý/khách hàng, hiểu nghiệp vụ, triển khai và hỗ trợ. CV chưa mô tả tên quy trình hoặc công cụ phối hợp BA/QA, nên không tự thêm Scrum, Jira hoặc vai trò quản lý.

Câu trả lời gắn với kinh nghiệm đã nêu:

"Trong các công việc tại Edsolab và Twenty-Tech, tôi có trao đổi với quản lý và khách hàng để hiểu yêu cầu, đề xuất cách xử lý và hỗ trợ sau triển khai. Khi làm chức năng tại TMSS/TYT, tôi cũng cần hiểu rõ quy tắc và ảnh hưởng dữ liệu trước khi triển khai.

Cách tôi tiếp cận là đưa yêu cầu về ví dụ cụ thể: ai thao tác, dữ liệu thuộc đơn vị nào, trạng thái nào được phép và trường hợp nào phải từ chối. Khi làm việc với BA, tôi dùng những ví dụ đó để xác nhận nghiệp vụ; khi phối hợp QA, tôi làm rõ luồng bình thường, trường hợp lặp và đồng thời."

Ví dụ luyện trao đổi theo TMSS: với một thay đổi quyết toán, cần xác định trạng thái được sửa, quyền người dùng, phạm vi đại lý, ảnh hưởng dữ liệu liên quan và cách xử lý khi gửi lại. Đây là cách phân tích minh họa, không khẳng định CV ghi một yêu cầu cụ thể như vậy.

Nếu được hỏi thói quen QA/khách hàng thực tế: kể đúng một lần bạn đã phối hợp, tên loại tài liệu sử dụng và cách xác nhận kết quả. Không nhận quy trình chưa từng tham gia.

1.10. Mục tiêu nghề nghiệp ba năm tới của bạn là gì?

Câu trả lời thống nhất mục tiêu trong CV:

"Mục tiêu ngắn hạn của tôi là trở thành lập trình viên cấp cao và một thành viên kỹ thuật nòng cốt. Về dài hạn, tôi hướng tới vai trò kiến trúc sư giải pháp. Trong ba năm tới, tôi muốn xây nền tảng cho hướng đó bằng kết quả công việc cụ thể.

Năm đầu, tôi tập trung hiểu nghiệp vụ MES, có thể tự chủ một luồng từ màn hình đến API và database, đồng thời hỗ trợ vận hành đúng quy trình.

Năm thứ hai, tôi muốn phụ trách phần phức tạp hơn như kiểm soát vật tư, tích hợp ERP hoặc truy xuất, và cải thiện năng lực hiệu năng, tính nhất quán và xử lý sự cố.

Đến năm thứ ba, tôi muốn có thể dẫn dắt kỹ thuật một phạm vi phù hợp, hỗ trợ đồng đội và tham gia quyết định giải pháp dựa trên yêu cầu và số liệu."

Ví dụ mục tiêu cụ thể: hoàn thành một luồng lệnh–tiêu hao–đồng bộ có tài liệu vận hành; hướng dẫn thành viên mới; giải thích được các đánh đổi về dữ liệu, tải và triển khai.

Không hứa chắc đạt chức danh kiến trúc sư sau ba năm. CV ghi đây là mục tiêu dài hạn; lộ trình trên tập trung năng lực cần xây dựng thay vì chỉ nhắm chức danh.

---

2. C# / .NET / ASP.NET Core

2.1. ⭐ Interface khác lớp trừu tượng như thế nào?

Interface mô tả một hợp đồng năng lực. **Lớp trừu tượng** mô tả một lớp cơ sở có thể chứa trạng thái, constructor và hành vi dùng chung. Một lớp C# chỉ kế thừa một lớp, nhưng có thể implement nhiều interface.

```
public interface IMaterialConsumer
{
    Task ConsumeAsync(Guid orderId, decimal quantity, CancellationToken ct);
}

public abstract class ProductionDevice
{
    protected ProductionDevice(string code) ⇒ Code = code;
    public string Code { get; }
    public abstract Task ConnectAsync(CancellationToken ct);
}
```

Ví dụ: API phụ thuộc IMaterialConsumer để không bị buộc vào cách lưu SQL cụ thể. Nhiều loại thiết bị có thể kế thừa ProductionDevice nếu thực sự có trạng thái và vòng đời chung.

Interface hiện đại có thể có implementation mặc định hoặc thành viên static theo khả năng ngôn ngữ; do đó "interface tuyệt đối không có code" không còn chính xác. Tuy nhiên, nó không cung cấp trạng thái instance và constructor như lớp cơ sở.

Liên hệ gRPC: interface trong Shared chỉ là hợp đồng. DI có thể trả về implementation nội bộ hoặc proxy gRPC. Tốc độ phụ thuộc đường thực thi, không phụ thuộc việc biến được khai báo bằng interface.

Liên hệ CV: dùng chính TYT làm ví dụ: interface trong Manager.Shared là hợp đồng; bạn đã nêu kinh nghiệm gRPC trong dự án. Khi hỏi "dùng Shared có nhanh hơn không", giải thích tốc độ khác nhau vì gọi implementation nội bộ hay proxy qua mạng, không vì riêng từ khóa interface.

2.2. SOLID gồm những nguyên tắc nào? Ví dụ thực tế

SOLID giúp giảm phụ thuộc và làm thay đổi có phạm vi rõ hơn:

Nguyên tắc	Ý nghĩa	Ví dụ MES
Trách nhiệm đơn nhất	Một thành phần có một lý do chính để thay đổi	Tách tính tiêu hao khỏi xuất báo cáo
Đóng với sửa đổi, mở với mở rộng	Thêm hành vi qua điểm mở rộng phù hợp	Thêm quy tắc tính hao hụt qua chiến lược
Thay thế Liskov	Implementation thay thế phải giữ hợp đồng hành vi	Repository không tự bỏ điều kiện tenant
Phân tách interface	Client chỉ phụ thuộc khả năng cần dùng	Tách đọc tồn khỏi điều chỉnh tồn
Đảo ngược phụ thuộc	Chính sách nghiệp vụ phụ thuộc abstraction	Dịch vụ lệnh phụ thuộc IInventoryRepository

Ví dụ: nếu mỗi loại sản phẩm có cách tính tiêu hao khác nhau, có thể dùng IMaterialConsumptionPolicy và từng implementation. Nhưng không nên tạo mười interface cho mười lớp CRUD không có nhu cầu thay thế.

Điểm cần giải thích: SOLID không phải số lượng layer hoặc số lượng interface. Một thiết kế có nhiều abstraction nhưng vẫn buộc sửa hàng loạt khi thay đổi nghiệp vụ chưa chắc đạt mục tiêu. Cần áp dụng vào nơi có biến động hoặc ranh giới trách nhiệm thực tế.

Ví dụ từ phạm vi CV: tác vụ nhắc lịch SMS/Zalo, xuất Excel nền và đồng bộ sự kiện có trách nhiệm khác nhau. Có thể dùng chúng để giải thích vì sao không nên nhét toàn bộ vào một controller. Đây là liên hệ nguyên tắc; CV không chứng minh

bạn đã tái cấu trúc hệ thống theo một mẫu SOLID cụ thể.

2.3. ⭐ Transient, Scoped và Singleton khác nhau thế nào?

Vòng đời	Khi tạo instance	Ví dụ phù hợp
Transient	Mỗi lần resolve	Dịch vụ tính toán nhẹ, không giữ trạng thái
Scoped	Một instance trong một DI scope	DbContext, đơn vị công việc theo request
Singleton	Một instance trong service provider	Dịch vụ dùng chung được thiết kế an toàn đồng thời

Trong HTTP thông thường, mỗi request có scope riêng. Hai lớp cùng resolve một scoped service trong request đó nhận cùng instance. Background worker không tự có scope HTTP; phải tạo scope khi cần.

```
builder.Services.AddTransient<ConsumptionCalculator>();
builder.Services.AddScoped<ProductionDbContext>();
builder.Services.AddSingleton<DeviceProtocolRegistry>();
```

Ví dụ lỗi: singleton worker giữ trực tiếp DbContext scoped sẽ giữ context quá lâu và dùng đồng thời không an toàn. Worker nên tạo scope cho mỗi đơn vị công việc rồi resolve dịch vụ trong scope.

Lưu ý: singleton không tự đồng nghĩa thread-safe; thread-safety do implementation quyết định. DbContext không dùng đồng thời cho nhiều thao tác. ASP.NET Core cho phép đăng ký nhiều lifetime, nhưng phải chọn theo trạng thái và dependency. Nguồn: vòng đời DI của ASP.NET Core.

Liên hệ TMSS: CV có worker nhắc lịch, dọn thông điệp và xuất Excel. Khi hỏi vòng đời DI, dùng một worker làm ví dụ: worker sống dài nhưng dịch vụ xử lý/DbContext cần scope cho đơn vị công việc. Nêu cách implementation thực tế tạo scope nếu bạn trực tiếp viết; không mặc định mọi worker cũ đã dùng cùng một cách.

2.4. IEnumerable khác IQueryable như thế nào?

IEnumerable hỗ trợ duyệt chuỗi phần tử. IQueryable bổ sung biểu thức truy vấn và provider có thể chuyển biểu thức sang SQL hoặc ngôn ngữ khác.

```
var query = db.ProductionOrders
    .Where(x => x.TenantId == tenantId && x.Status == OrderStatus.Running)
    .Select(x => new { x.Id, x.Code });

var page = await query.Take(20).ToListAsync(ct);
```

Trong ví dụ EF Core, lọc và giới hạn được provider dịch sang SQL. Nếu gọi ToList trước rồi mới Where, việc lọc sau đó chạy trên dữ liệu đã tải về.

Phân biệt quan trọng: IEnumerable không nhất thiết đã nằm hoàn toàn trong bộ nhớ; có thể thực thi trì hoãn hoặc đọc từng phần. AsEnumerable không lập tức tải hết dữ liệu, nhưng các toán tử LINQ sau đó dùng cơ chế LINQ trên IEnumerable.

Ví dụ lỗi: tải một triệu lịch sử vào List rồi lọc theo nhà máy. Cách này tăng truyền mạng và RAM; điều kiện nhà máy nên nằm trong truy vấn database.

Không trả IQueryable ra mọi layer một cách vô điều kiện: caller có thể vô tình phá ranh giới dữ liệu hoặc tạo truy vấn khó kiểm soát.

Liên hệ OESMedicalAPI: CV ghi tìm sản phẩm bằng truy vấn động. Bạn có thể giải thích việc kết hợp điều kiện tenant, thuộc tính sản phẩm và phân trang trước khi materialize. Cơ chế là LINQ hay SQL động phải nói theo code thật; không suy ra từ chữ "truy vấn động" rằng dự án chắc chắn trả IQueryable qua các layer.

2.5. async/await và Task hoạt động thế nào trong C#?

Task đại diện cho một hoạt động đang hoặc sẽ hoàn tất; nó có thể có kết quả, lỗi hoặc bị hủy. async cho phép compiler tạo cơ chế tiếp tục thực thi quanh await.

Khi await một thao tác chưa hoàn tất, phương thức có thể nhường quyền điều khiển cho caller. Với I/O bất đồng bộ phù hợp, không cần giữ một thread chỉ để chờ database hoặc mạng. async không tự tạo thread mới, và phần code trước await chưa hoàn tất vẫn chạy đồng bộ.

```
public async Task<OrderDto?> GetAsync(Guid id, CancellationToken ct)
{
    var order = await db.ProductionOrders
        .AsNoTracking()
        .Where(x ⇒ x.Id == id)
        .Select(x ⇒ new OrderDto(x.Id, x.Code))
        .SingleOrDefaultAsync(ct);

    return order;
}
```

Ví dụ: 500 request chờ database không nên cần 500 thread bị chặn bằng Result. Dùng await giúp server sử dụng thread hiệu quả hơn, nhưng không làm truy vấn SQL tự chạy nhanh hơn.

Tránh async void ngoài event handler; truyền CancellationToken; không bỏ quên Task cần quan sát. CPU tính toán nặng vẫn cần CPU thật dù method có async. Nguồn: các tình huống lập trình bất đồng bộ C#.

2.6. Task.WhenAll khác Task.Run như thế nào?

Task.WhenAll tạo Task hoàn tất khi mọi Task đầu vào hoàn tất; bản thân nó không tự khởi chạy công việc hay cấp thread riêng. **Task.Run** đưa delegate lên ThreadPool, thường dùng để chuyển công việc CPU ra khỏi thread giao diện.

```
var orderTask = orderClient.GetAsync(orderId, ct);
var machineTask = machineClient.GetStatusAsync(machineId, ct);
await Task.WhenAll(orderTask, machineTask);
```

Hai lời gọi độc lập được bắt đầu trước khi await chung. Nếu gọi tuần tự và await từng lời gọi ngay, tổng thời gian chờ có thể dài hơn.

Ví dụ Task.Run: ứng dụng desktop tính toán tối ưu lịch sản xuất để tránh khóa giao diện. Trong ASP.NET Core, bọc truy vấn database async bằng Task.Run thường chỉ tăng điều phối, không tăng khả năng xử lý.

Bẫy: không chạy hai truy vấn đồng thời trên cùng DbContext. Không tạo 100.000 Task I/O một lúc; giới hạn số tác vụ đang chạy. WhenAll không tự hủy tác vụ còn lại khi một tác vụ lỗi. Task tổng lưu thông tin lỗi; nếu cần điều tra toàn bộ lỗi, phải xem các task tương ứng. Nguồn: lập trình bất đồng bộ dựa trên Task.

2.7. ⭐ Giải thích chuỗi middleware trong ASP.NET Core

Middleware là các thành phần xử lý request theo thứ tự đăng ký. Thành phần có thể xử lý trước, gọi next, xử lý sau hoặc kết thúc request sớm. Khi các thành phần await next, chiều response đi ngược lại.

```
app.UseExceptionHandler();
app.UseHttpsRedirection();
app.UseRouting();
app.UseCors("FactoryUi");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Đây là thứ tự minh họa, không phải đề nghị thay pipeline hiện tại. Ứng dụng sau reverse proxy còn cần cấu hình forwarded headers đúng vị trí và proxy được tin cậy.

Ví dụ: request xác nhận công đoạn đến middleware ghi log, sau đó xác thực danh tính, kiểm tra quyền và chạy controller. Nếu chưa đăng nhập, request có thể kết thúc trước controller. Exception handler đặt phía ngoài để bắt lỗi từ các thành phần phía sau, khi response còn có thể ghi.

Bẫy: authorization trước authentication có thể khiến chính sách không nhận được danh tính mong muốn. Middleware đã gửi response không nên tiếp tục ghi response khác. Exception từ background worker cũng không được HTTP middleware tự bắt. Nguồn: pipeline ASP.NET Core.

2.8. Authentication khác Authorization thế nào?

Xác thực trả lời "bạn là ai?". **Phân quyền** trả lời "bạn được làm gì với tài nguyên này?".

Ví dụ, token hợp lệ xác nhận người dùng A. Nhưng A chỉ được xem lệnh của nhà máy HN và không được duyệt phiếu hủy tại nhà máy HCM.

```
[Authorize(Policy = "ProductionOrder.Release")]
[HttpPost("{id:guid}/release")]
public Task<IActionResult> Release(Guid id, CancellationToken ct)
    ⇒ releaseHandler.ExecuteAsync(id, User, ct);
```

Ngoài policy ở endpoint, handler phải kiểm tra tài nguyên thuộc tenant/nhà máy mà người dùng được phép truy cập. Quyền chung "được xem lệnh" không tự cho phép xem mọi lệnh.

Mã phản hồi: 401 thường dùng khi thiếu hoặc không có thông tin xác thực hợp lệ; 403 khi đã xác thực nhưng không đủ quyền. Tùy chính sách che giấu tài nguyên, có thể trả 404 cho tài nguyên không được phép thấy.

Ẩn nút ở frontend chỉ cải thiện trải nghiệm. Backend luôn kiểm tra quyền vì người dùng có thể gọi API trực tiếp. Không tin TenantId hoặc FactoryId chỉ vì client gửi lên.

Câu nói theo CV: "Ở TMSS tôi triển khai MasterSSO, phân quyền theo vai trò và cô lập dữ liệu Host/đại lý. Với TYT, CV của tôi có phần đăng nhập hợp nhất tích hợp Microsoft Entra ID, Auth0, Google, Apple ID và Zalo OA. Tôi phân biệt rõ việc nhận diện người dùng với việc cho phép họ thao tác trên một tài nguyên."

Khi hỏi sâu, chỉ rõ bên nào phát hành token và ứng dụng nào tiêu thụ trong luồng bạn phụ trách. Các nhà cung cấp không nhất thiết dùng cùng một giao thức hoặc kiểu token; không nói mọi tích hợp Zalo OA đều là OIDC. Mã hóa token cũng không thay thế kiểm tra quyền.

2.9. Xử lý ngoại lệ tập trung và ProblemDetails thế nào?

Mục tiêu là trả lỗi nhất quán, ánh xạ đúng ý nghĩa, có mã truy vết và không lộ stack trace hoặc secret.

```
builder.Services.AddProblemDetails();
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();

var app = builder.Build();
app.UseExceptionHandler();
```

GlobalExceptionHandler là implementation IExceptionHandler của ứng dụng. Có thể ánh xạ lỗi dữ liệu không hợp lệ thành 400/422 theo hợp đồng, xung đột phiên bản thành 409, tài nguyên thiếu thành 404 và lỗi ngoài dự kiến thành 500.

Ví dụ response minh họa:

```
{
  "title": "Không đủ vật tư",
  "status": 409,
  "detail": "Lượng khả dụng không đáp ứng yêu cầu.",
  "traceId": "ma-truy-vet-minh-hoa"
}
```

Log nội bộ lưu exception và ngữ cảnh đã kiểm soát; response chỉ chứa thông tin người dùng cần. Hủy request từ client không nên bị ghi như lỗi hệ thống nghiêm trọng một cách máy móc.

Liên hệ repo: TYT hiện có ProcessRequest/HandlerException và BaseResponse. Đây là phương án cho ứng dụng thiết kế mới; không tự kết luận repo đã dùng ProblemDetails hoặc đổi hợp đồng cũ mà không đánh giá consumer. Nguồn: xử lý lỗi ASP.NET Core.

2.10. ⭐ Thiết kế REST API quản lý lệnh sản xuất thế nào?

Trước hết xác định tài nguyên, trạng thái, thao tác hợp lệ và quyền. Lệnh có thể đi từ Nháp → Phát hành → Đang chạy → Hoàn tất; có nhánh tạm dừng, hủy theo quy tắc.

Nhu cầu	Endpoint minh họa
Tạo lệnh	POST /api/production-orders
Danh sách có lọc và phân trang	GET /api/production-orders
Chi tiết lệnh	GET /api/production-orders/{id}
Sửa phần được phép	PATCH /api/production-orders/{id}
Phát hành lệnh	POST /api/production-orders/{id}/release
Ghi nhận kết quả công đoạn	POST /api/production-orders/{id}/operation-results

Tạo thành công trả 201 và địa chỉ tài nguyên; tác vụ chấp nhận xử lý nền có thể trả 202 kèm cách theo dõi. Thao tác đã hoàn thành đồng bộ không nên trả 202 chỉ vì có phát message phụ.

Ví dụ: khi phát hành, backend kiểm tra BOM/routing có hiệu lực, quyền nhà máy và phiên bản lệnh. ETag/If-Match hoặc rowversion giúp phát hiện sửa trên dữ liệu cũ. Khóa idempotency giúp request tạo lệnh được gửi lại không tạo hai lệnh.

Không cho client PATCH trực tiếp Status thành Hoàn tất để bỏ qua quy tắc. Ghi nhận hành động nghiệp vụ, audit, lỗi và ràng buộc trạng thái rõ ràng.

2.11. Kiểm soát đồng thời lạc quan và bi quan khác nhau thế nào?

Lạc quan: đọc không giữ khóa dài, khi ghi kiểm tra phiên bản còn giống lúc đọc. **Bi quan:** lấy khóa phù hợp để các giao dịch khác không cùng thay đổi tài nguyên theo cách xung đột.

```
UPDATE dbo.ProductionOrder
SET PlannedQuantity = @PlannedQuantity
WHERE Id = @Id
  AND TenantId = @TenantId
  AND RowVersion = @ExpectedRowVersion;
```

Nếu không cập nhật được dòng nào, phải phân biệt tài nguyên không tồn tại, sai tenant hoặc phiên bản đã thay đổi. Trong SQL Server, rowversion là dấu phiên bản nhị phân tự tăng ở cấp database, không phải thời gian.

Ví dụ lạc quan: hai kế hoạch viên sửa cùng lệnh; người thứ hai được thông báo tải lại và quyết định hợp nhất.

Ví dụ bi quan: nghiệp vụ phải đọc nhiều dòng tồn rồi phân bổ nguyên tử; có thể cần khóa trong một transaction ngắn. Khóa phải giữ đúng transaction, thứ tự truy cập thống nhất.

Không giữ transaction database trong lúc chờ người dùng bấm xác nhận hoặc gọi ERP qua mạng. Với phép trừ tồn đơn giản, UPDATE có điều kiện thường rõ hơn vòng đọc–kiểm tra–ghi.

2.12. Triển khai validation và logging thế nào?

Validation nên có nhiều lớp, mỗi lớp giải quyết một loại lỗi:

1. Hình dạng request: trường bắt buộc, độ dài, định dạng, số lượng dương.
2. Nghiệp vụ: công đoạn đã được phát hành, đúng vật tư, lô không bị cách ly.
3. Database: khóa ngoại, unique key, CHECK và giao dịch bảo vệ bất biến.

Ví dụ: frontend chặn Quantity bằng 0 để phản hồi nhanh; API vẫn kiểm tra lại; database dùng ràng buộc phù hợp. Kiểm tra tồn bằng SELECT trước UPDATE không đủ chống hai request đồng thời.

Logging dùng cấu trúc và định danh:

```
logger.LogInformation(
    "Ghi nhận tiêu hao {ProductionOrderId}, {MaterialId}, {RequestId}",
    orderId, materialId, requestId);
```

Nên có TenantId, TraceId và mã nghiệp vụ khi phù hợp, nhưng không ghi token, mật khẩu hoặc toàn bộ payload chứa dữ liệu nhạy cảm. Phân biệt log chẩn đoán với audit nghiệp vụ: audit phải được lưu tin cậy theo yêu cầu giao dịch.

Trong TYT, validation có pattern gọi thủ công ở action. Khi phỏng vấn, phân biệt cách repo đang làm với cách bạn đề xuất cho ứng dụng mới.

Liên hệ CV: FluentValidation được liệt kê ở TMSS; Serilog và Elasticsearch ở TYT. Bạn có thể trình bày kiểm tra request bằng validator, quy tắc ở tầng nghiệp vụ và log có mã tương quan. CV không xác nhận đã có ProblemDetails toàn hệ thống; không nhận đó là thành phần đang dùng nếu API vẫn trả BaseResponse.

---

3. SQL Server / EF Core / giao dịch

3.1. ⭐ Chỉ mục clustered và non-clustered khác nhau thế nào?

Với bảng rowstore thông thường, clustered index có tầng lá chứa các dòng dữ liệu; một bảng chỉ có một clustered index. Non-clustered index là cấu trúc riêng, chứa khóa và thông tin định vị dòng; bảng có thể có nhiều chỉ mục loại này.

Ví dụ minh họa:

```
CREATE INDEX IX_ProductionOrder_Tenant_Status_Created
ON dbo.ProductionOrder (TenantId, Status, CreatedOnUtc, Id)
INCLUDE (Code, PlannedQuantity);
```

Chỉ mục này hướng đến truy vấn lọc tenant và trạng thái, rồi lấy lệnh theo thời điểm. INCLUDE bổ sung cột kết quả mà không đưa chúng vào toàn bộ khóa sắp xếp.

Đánh đổi: mỗi INSERT/UPDATE/DELETE có thể phải bảo trì thêm index; quá nhiều index làm ghi chậm và tăng dung lượng. Chọn thứ tự khóa theo điều kiện lọc, sắp xếp và phân bố dữ liệu, không chỉ theo một quy tắc "cột nhiều giá trị nhất luôn đứng đầu".

Bẫy: khóa chính không bắt buộc là clustered. Có clustered index cũng không bảo đảm kết quả SELECT đã được sắp xếp; vẫn cần ORDER BY. Nguồn: chỉ mục SQL Server.

3.2. INNER JOIN khác LEFT JOIN như thế nào?

INNER JOIN chỉ giữ các cặp có điều kiện nối khớp. LEFT JOIN giữ mọi dòng bên trái; khi không có dòng bên phải, các cột tương ứng là NULL.

Ví dụ: cần liệt kê mọi lệnh sản xuất, kể cả chưa có kết quả:

```
SELECT o.Id, r.GoodQuantity
FROM dbo.ProductionOrder AS o
LEFT JOIN dbo.OperationResult AS r
  ON r.ProductionOrderId = o.Id
 AND r.TenantId = o.TenantId
WHERE o.TenantId = @TenantId;
```

Nếu đổi thành INNER JOIN, lệnh chưa có kết quả sẽ biến mất.

Bẫy phổ biến: thêm WHERE r.GoodQuantity > 0 sẽ loại các dòng NULL; với điều kiện này, mục tiêu giữ lệnh chưa có kết quả không còn đạt được. Nếu muốn chỉ nối các kết quả tốt nhưng vẫn giữ mọi lệnh, đưa điều kiện phù hợp vào ON.

Một lệnh có năm kết quả sẽ tạo năm dòng sau JOIN. Nếu cần tổng theo lệnh, phải GROUP BY hoặc tổng hợp bảng kết quả trước; đừng dùng DISTINCT để che việc sai cấp độ dữ liệu. Với hệ đa tenant, điều kiện nối cũng phải giữ ranh giới tenant.

3.3. Phát hiện và tối ưu truy vấn SQL chậm thế nào?

Tôi xác định truy vấn, tham số, dữ liệu và thời điểm bị chậm trước khi sửa.

1. Đo thời gian toàn API, thời gian database và số lần gọi; kiểm tra có N+1 hay không.
2. Dùng Query Store hoặc công cụ theo dõi để tìm truy vấn và kế hoạch liên quan.
3. Xem execution plan thực tế ở môi trường phù hợp: sai ước lượng số dòng, scan lớn, sort, lookup nhiều hoặc spill.
4. Kiểm tra waits/blocking: truy vấn chờ khóa không nhất thiết thiếu index.
5. Xem index, thống kê, cách viết điều kiện và số cột/dòng trả về.

Ví dụ: lọc theo ngày bằng CAST(CreatedOnUtc AS date) có thể làm việc tận dụng index khó hơn. Thường dùng khoảng:

```
WHERE CreatedOnUtc ≥ @FromUtc
  AND CreatedOnUtc < @ToUtc
```

Sau sửa, so sánh logical reads, CPU, thời gian và kết quả trên dữ liệu đại diện, gồm cả tham số nhiều/ít dữ liệu. Index seek không luôn tốt hơn scan; tùy số dòng cần đọc.

Không dùng NOLOCK như cách chữa chung: nó có thể đọc dữ liệu chưa commit hoặc kết quả thiếu/trùng, không phù hợp quyết định tồn kho. Không tự xóa cache kế hoạch toàn server production để thử.

Ví dụ nên chuẩn bị từ dự án: một truy vấn danh sách phụ tùng/báo giá trong TMSS hoặc lịch sử điểm trong TYT mà bạn trực tiếp làm. Ghi SQL, tham số, index và số liệu trước/sau nếu có. CV chỉ nói làm các nghiệp vụ và dữ liệu lớn, không có số đo hiệu năng để tự điền kết quả cải thiện.

3.4. ⭐ ACID trong giao dịch database là gì?

Tính chất	Ý nghĩa	Ví dụ tiêu hao vật tư
Nguyên tử	Cùng thành công hoặc cùng rollback	Trừ tồn và ghi lịch sử không được lệch nhau
Nhất quán	Giữ các ràng buộc hợp lệ	Không để lượng khả dụng âm
Cô lập	Kiểm soát tương tác giao dịch đồng thời	Hai request không cùng tiêu phần tồn không đủ
Bền vững	Commit có tính bền vững theo cấu hình lưu trữ	Giao dịch đã xác nhận có thể phục hồi sau sự cố trong phạm vi cam kết

Ví dụ: kho có 10 kg. Lệnh cần 4 kg. Giao dịch gồm cập nhật tồn còn 6, thêm phiếu tiêu hao và thêm Outbox. Nếu thêm phiếu lỗi thì phải rollback cập nhật tồn.

Điểm quan trọng: mở transaction không tự làm một thuật toán đọc–kiểm tra–ghi trở nên đúng dưới mọi mức cô lập. Vẫn cần câu lệnh cập nhật có điều kiện, khóa hoặc kiểm tra phiên bản phù hợp.

Commit SQL không đồng nghĩa đã commit RabbitMQ hay ERP. ACID cục bộ không tự mở rộng qua mạng; đó là lý do cần thiết kế Outbox, idempotency và đối soát ở cấp hệ thống.

3.5. Các mức cô lập giao dịch trong SQL Server là gì?

Mức/cơ chế	Ý nghĩa chủ yếu	Hạn chế cần hiểu
READ UNCOMMITTED	Có thể đọc thay đổi chưa commit	Kết quả không đáng tin cho quyết định nghiệp vụ
READ COMMITTED dùng khóa	Không đọc dữ liệu chưa commit	Đọc lại có thể thấy thay đổi và dòng mới
READ COMMITTED với RCSI	Mỗi câu lệnh đọc ảnh đã commit theo phiên bản	Hai SELECT trong một transaction có thể thấy hai thời điểm khác nhau
REPEATABLE READ	Giữ các dòng đã đọc khỏi bị sửa/xóa trong giao dịch	Vẫn có thể có dòng mới thỏa điều kiện
SNAPSHOT	Đọc theo ảnh nhất quán của giao dịch	Có thể xung đột ghi, cần xử lý retry
SERIALIZABLE	Bảo vệ cả phạm vi đọc trước phantom	Có thể tăng blocking và giảm mức đồng thời

RCSI là tùy chọn database làm thay đổi hành vi READ COMMITTED, không phải một tên isolation level độc lập trong câu lệnh SET TRANSACTION ISOLATION LEVEL.

Ví dụ: tính tổng tồn bằng hai truy vấn dưới RCSI không tự bảo đảm hai truy vấn nhìn cùng snapshot giao dịch. Nếu nghiệp vụ cần tính nhất quán đó, cần chọn cơ chế phù hợp.

Không mặc định chọn SERIALIZABLE cho toàn bộ ứng dụng. Hãy bảo vệ bất biến bằng transaction ngắn và ràng buộc đúng. Nguồn: khóa và phiên bản dòng SQL Server.

3.6. ⭐ Deadlock là gì? Phát hiện và xử lý thế nào?

Deadlock xảy ra khi các tác vụ tạo vòng chờ tài nguyên. Blocking thông thường là chờ một tác vụ khác; deadlock có chu trình khiến chúng không tự tiến được.

Ví dụ: giao dịch A khóa vật tư M1 rồi cần M2; B khóa M2 rồi cần M1. SQL Server chọn một nạn nhân và rollback giao dịch đó, thường báo lỗi 1205.

Điều tra: xem deadlock graph từ Extended Events, chẳng hạn dữ liệu system_health còn lưu, xác định câu lệnh, index, loại khóa và thứ tự truy cập. Không kết luận chỉ từ thông báo timeout.

Khắc phục theo nguyên nhân:

- Truy cập vật tư theo thứ tự thống nhất.
- Rút ngắn transaction; không gọi dịch vụ ngoài khi đang giữ khóa.
- Tối ưu điều kiện/index để giảm phạm vi khóa.
- Chọn isolation phù hợp; row versioning có thể giảm một số xung đột đọc–ghi nhưng không loại mọi deadlock.
- Retry toàn bộ đơn vị giao dịch bị rollback, có giới hạn và khoảng chờ ngẫu nhiên.

Ví dụ: phân bổ ba vật tư thì sắp xếp theo khóa ổn định trước khi cập nhật. Retry phải dựa trên nghiệp vụ idempotent, không tự lặp tác dụng phụ đã gửi ra ngoài transaction. Nguồn: hướng dẫn deadlock SQL Server.

3.7. Stored procedure khác view như thế nào?

View là một truy vấn đặt tên, dùng như nguồn dữ liệu trong SELECT. View thông thường không lưu sẵn toàn bộ kết quả và không nhận tham số theo cách procedure nhận.

Stored procedure là chương trình T-SQL được gọi bằng EXEC, có tham số, câu lệnh điều khiển, thao tác ghi và quản lý transaction khi cần.

Ví dụ: view thể hiện các lệnh cùng tên sản phẩm để nhiều báo cáo dùng chung. Procedure ghi nhận tiêu hao nhận TenantId, OrderId, MaterialId, Quantity và RequestId rồi cập nhật nguyên tử.

Có indexed view với cơ chế lưu và ràng buộc riêng; do đó không nên nói mọi view đều hoàn toàn không lưu dữ liệu. Một số view có thể cập nhật theo điều kiện, nên "view tuyệt đối chỉ đọc" cũng không chính xác.

Hiệu năng: procedure không tự nhanh hơn câu SQL có tham số từ ứng dụng. Execution plan, index, tham số và lượng dữ liệu quan trọng hơn hình thức đóng gói.

Trong TYT, nhiều truy vấn repository gọi procedure bằng Dapper. Khi sửa procedure, cần hiểu caller, danh sách result set, kiểu dữ liệu và transaction, không chỉ nội dung SQL.

3.8. ROW_NUMBER, RANK và DENSE_RANK khác nhau thế nào?

Cả ba đánh số trong cửa sổ có ORDER BY, nhưng xử lý đồng hạng khác nhau:

Sản lượng	ROW_NUMBER	RANK	DENSE_RANK
100	1	1	1
100	2	1	1
90	3	3	2
80	4	4	3

ROW_NUMBER tạo số riêng từng dòng; RANK có khoảng trống sau đồng hạng; DENSE_RANK không có khoảng trống.

Ví dụ: xếp hạng dây chuyền theo số lượng đạt thì dùng RANK/DENSE_RANK nếu hai dây chuyền bằng nhau phải cùng hạng. Lấy bản ghi mới nhất cho từng thiết bị có thể dùng:

```
ROW_NUMBER() OVER (
    PARTITION BY TenantId, DeviceId
    ORDER BY OccurredOnUtc DESC, Id DESC
) AS rn
```

Sau đó lọc rn = 1 ở truy vấn ngoài.

Bẫy: nếu ORDER BY không phân biệt hoàn toàn các dòng, thứ tự của ROW_NUMBER giữa các dòng bằng nhau không được bảo đảm. Thêm khóa ổn định khi cần chọn một dòng nhất quán. Với RANK, thêm Id vào tiêu chí xếp hạng sẽ làm mất ý nghĩa đồng hạng theo sản lượng.

3.9. ⭐ EF Core khác Dapper thế nào? Khi nào dùng từng loại?

EF Core là ORM có LINQ, mapping quan hệ, change tracking, quản lý migration và cơ chế concurrency token. Dapper là công cụ ánh xạ kết quả SQL sang object, cho phép chủ động viết SQL hoặc gọi stored procedure.

Tiêu chí	EF Core	Dapper
Viết truy vấn	LINQ hoặc SQL	SQL/procedure chủ động
Theo dõi thay đổi	Có, tùy cấu hình	Không có cơ chế tracking tương đương
Mapping quan hệ	Tích hợp nhiều khả năng	Thường tự tổ chức rõ hơn
Migration schema	Có công cụ hỗ trợ	Cần công cụ/quy trình khác
Phù hợp	CRUD, aggregate, phát triển nghiệp vụ	Truy vấn phức tạp, báo cáo, hệ thống nhiều procedure

Ví dụ: CRUD danh mục vật tư dùng EF Core để phát triển thuận tiện; báo cáo traceability nhiều cấp dùng SQL tối ưu qua Dapper nếu phù hợp.

Không khẳng định Dapper luôn nhanh hơn theo mức cố định. Phải so SQL thực thi và đo workload. EF Core dùng projection, no-tracking hợp lý có thể đáp ứng tốt.

Có thể kết hợp nhưng nếu cùng một nghiệp vụ cần nguyên tử, phải dùng đúng connection/transaction chung; hai công nghệ không tự chia sẻ transaction.

Câu trả lời theo CV: "Trong TMSS tôi làm với ABP/ASP.NET Zero, EF Core và Dapper; TYT cũng có Dapper và EF Core trong stack. Tôi chọn công cụ theo nhu cầu truy cập dữ liệu và cách dự án tổ chức, thay vì coi một công cụ luôn tốt hơn."

Nếu muốn kể rằng CRUD dùng EF và báo cáo dùng Dapper, hãy chọn một ví dụ thật. Đó là cách phân chia thường gặp, chưa tự là bằng chứng dự án của bạn áp dụng đúng cách đó ở mọi chức năng.

3.10. N+1 trong EF Core là gì?

N+1 là tình huống một truy vấn lấy N đối tượng rồi phát sinh thêm một truy vấn cho mỗi đối tượng. Thường gặp khi lazy loading hoặc gọi repository bên trong vòng lặp.

Ví dụ: lấy 100 lệnh rồi truy vấn công đoạn của từng lệnh tạo khoảng 101 lượt gọi database.

```
var orders = await db.ProductionOrders
    .Where(x ⇒ x.TenantId == tenantId)
    .Select(x ⇒ new OrderSummary(
        x.Id,
        x.Code,
        x.Operations.Count()))
    .Take(50)
    .ToListAsync(ct);
```

Projection chỉ lấy dữ liệu cần và để provider xử lý tổng hợp. Tùy nhu cầu, dùng Include, tải nhóm theo danh sách khóa hoặc truy vấn chuyên biệt.

Đánh đổi: Include nhiều collection có thể làm số dòng JOIN tăng mạnh. Split query giảm một số vấn đề nhưng có thêm lượt gọi và cần cân nhắc tính nhất quán giữa truy vấn.

Cách phát hiện: bật log database phù hợp ở môi trường kiểm tra, đếm số command cho một request và xem thời gian. Không chỉ nhìn thời gian chạy trên database rất ít dữ liệu. Nguồn: truy vấn hiệu quả với EF Core.

3.11. ⭐ ROWLOCK, UPDLOCK và READPAST là gì?

- **ROWLOCK:** yêu cầu khóa ở mức dòng/khóa index khi phù hợp; không bảo đảm không bao giờ xảy ra lock escalation.
- **UPDLOCK:** dùng update lock cho dữ liệu đọc, giữ đến hết transaction, hữu ích cho thao tác đọc rồi cập nhật.
- **READPAST:** bỏ qua dòng bị khóa theo điều kiện áp dụng; không bỏ qua mọi page lock.

Ví dụ: nhiều worker tranh nhận công việc từ bảng queue có thể dùng cơ chế này để giảm chờ nhau. Nhưng SELECT lấy việc rồi UPDATE riêng rẽ vẫn có khoảng tranh chấp nếu không được bảo vệ đúng.

Lưu ý cấu hình: READPAST có giới hạn theo isolation level và RCSI. Không copy cùng bộ hint sang mọi database; có trường hợp cần chọn khóa đọc tường minh phù hợp, đồng thời tránh tổ hợp hint xung đột.

READPAST cũng có thể khiến việc bị khóa liên tục bị trì hoãn. Cần giám sát tuổi bản ghi chưa xử lý. Nguồn: bảng hint chính thức của SQL Server.

Liên hệ repo: procedure LoyaltyOutboxMessage_Claim và LoyaltyInboxMessage_Claim sử dụng bộ hint này. Source cho thấy cơ chế claim; chưa chứng minh toàn bộ xử lý có khả năng chống lặp khi lease hết hạn.

Liên hệ CV và mã nguồn: CV TMSS nhắc ROWLOCK/READPAST. Trong source TYT đã đọc có procedure claim dùng thêm UPDLOCK. Đây là hai căn cứ riêng; không dùng một procedure của TYT để nói chắc rằng mọi queue TMSS có cùng SQL và

cùng cấu hình database.

3.12. Ngăn trừ tồn trùng dưới nhiều request đồng thời thế nào?

Phải giải quyết hai vấn đề độc lập: **cùng request được gửi lại** và **nhiều request khác nhau cùng tranh tồn**.

Thiết kế khóa duy nhất theo phạm vi nghiệp vụ, ví dụ (TenantId, RequestId) cho phiếu tiêu hao. Lưu fingerprint của request để phát hiện cùng khóa nhưng payload khác. Toàn bộ nhận request, cập nhật tồn, ghi sổ và Outbox thuộc cùng transaction.

SQL minh họa đơn vị giao dịch độc lập; các tham số đã được xác thực:

```
SET XACT_ABORT ON;
BEGIN TRY
    BEGIN TRANSACTION;

    IF @Quantity ≤ 0
        THROW 50001, N'Số lượng phải lớn hơn không', 1;

    -- Unique key (TenantId, RequestId) phải tồn tại trên bảng này.
    INSERT dbo.MaterialConsumption
        (Id, TenantId, RequestId, ProductionOrderId, MaterialId, Quantity)
    VALUES
        (@Id, @TenantId, @RequestId, @ProductionOrderId, @MaterialId, @Quantity);

    UPDATE dbo.InventoryBalance
    SET AvailableQuantity = AvailableQuantity - @Quantity
    WHERE TenantId = @TenantId
      AND WarehouseId = @WarehouseId
      AND MaterialId = @MaterialId
      AND LotId = @LotId
      AND AvailableQuantity ≥ @Quantity;

    IF @@ROWCOUNT <> 1
        THROW 50002, N'Không đủ vật tư hoặc không có dòng tồn hợp lệ', 1;

    -- Ghi sổ biến động và Outbox tại đây, cùng transaction.
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
```

Ví dụ: có 10 kg, hai request khác nhau cùng cần 7 kg; chỉ một UPDATE được thành công. Cùng RequestId gửi lại bị unique key chặn trước khi trừ thêm.

Ứng dụng chỉ xử lý lỗi trùng của đúng ràng buộc idempotency, rồi đọc kết quả đã commit và so fingerprint; không biến mọi lỗi unique thành thành công. Không dùng SELECT kiểm tra tồn hoặc khóa C# trong một process làm bảo vệ duy nhất.

Ví dụ này giả định một dòng tồn duy nhất cho khóa kho–vật tư–lô. Với giữ chỗ hoặc phân bổ nhiều lô, phải điều chỉnh mô hình và quy tắc cập nhật tương ứng.

Liên hệ kinh nghiệm: Booking API trong CV có giữ dịch vụ và hủy khi hết thời gian; TYT có hạn mức voucher. Dùng hai ví dụ này để giải thích sự khác nhau giữa chống gửi lặp và cạnh tranh tài nguyên. Không chuyển nguyên thuật toán đặt chỗ sang MES mà bỏ điều kiện lô, chất lượng hoặc đơn vị tính.

3.13. Phân trang bảng hàng triệu dòng thế nào?

OFFSET/FETCH dễ dùng và hỗ trợ nhảy trang, nhưng trang sâu có thể phải bỏ qua nhiều dòng. Khi người dùng chủ yếu xem tiếp, dùng phân trang theo khóa tiếp nối.

```
SELECT TOP (@PageSize) Id, Code, CreatedOnUtc
FROM dbo.ProductionOrder
WHERE TenantId = @TenantId
  AND (
      CreatedOnUtc < @LastCreatedOnUtc
      OR (CreatedOnUtc = @LastCreatedOnUtc AND Id < @LastId)
  )
ORDER BY CreatedOnUtc DESC, Id DESC;
```

Trang đầu không dùng điều kiện cursor. Index cần hỗ trợ tenant và thứ tự khóa. Cursor chứa cả thời điểm và Id, gắn với bộ lọc đang dùng; backend kiểm tra hợp lệ và giới hạn PageSize.

Ví dụ: xem lịch sử sản xuất từ mới đến cũ. Request sau tiếp tục sau dòng cuối của trang trước, thay vì bỏ qua hàng trăm nghìn dòng.

Đánh đổi: không thuận tiện nhảy trực tiếp trang 50.000. Việc cập nhật/xóa dữ liệu vẫn có thể ảnh hưởng kết quả; nếu cần ảnh chụp ổn định cho xuất báo cáo, phải xác định thêm mốc snapshot. Đếm tổng chính xác cũng có chi phí riêng. Nguồn: phân trang EF Core.

3.14. Thiết kế bảng lịch sử sản xuất và traceability thế nào?

Tách trạng thái hiện tại khỏi dữ liệu lịch sử có liên kết:

Bảng minh họa	Nội dung chính
ProductionOrder	Lệnh, sản phẩm, số lượng, trạng thái
ProductionOperation	Công đoạn, thứ tự, trạm làm việc
OperationExecution	Lần thực hiện, người/máy, bắt đầu/kết thúc
MaterialLot	Vật tư, mã lô, nhà cung cấp, trạng thái chất lượng
MaterialConsumption	Lô đầu vào, lượng, đơn vị, công đoạn
ProductUnit hoặc OutputLot	Sản phẩm hoặc lô đầu ra
GenealogyLink	Quan hệ đầu vào–đầu ra và lượng phân bổ
QualityInspection	Kết quả kiểm tra và quyết định xử lý
ProductionEvent	Lịch sử hành động, phiên bản và nguồn

Ví dụ: lô nhựa NL-01 cấp vào công đoạn ép, tạo lô TP-09. GenealogyLink nối hai lô; nếu sản xuất theo serial thì phải nối đúng serial hoặc phạm vi phân bổ đủ chi tiết.

Mọi bảng liên quan cần TenantId/nhà máy phù hợp, mã nghiệp vụ và định danh kỹ thuật, thời gian sự kiện và thời gian nhận khi cần. Lưu phiên bản BOM/routing hoặc snapshot đã dùng.

Không chỉ lưu ghi chú "dùng vật tư A": thiếu lô, lượng và quan hệ đầu ra thì không truy ngược chính xác được. Lưu lịch sử theo kiểu bổ sung và tạo bản ghi điều chỉnh thay vì âm thầm sửa bằng chứng cũ.

Liên hệ OESMedicalAPI: bạn đã nêu lịch sử cập nhật sản phẩm. Kinh nghiệm đó giúp giải thích trước/sau, người sửa và thời điểm. Traceability sản xuất còn cần liên kết vật tư đầu vào với đầu ra; lịch sử sửa một sản phẩm trong danh mục chưa đủ cho truy xuất lô/serial.

---

4. RabbitMQ / hệ phân tán / đồng thời

4.1. ⭐ Vì sao dùng RabbitMQ trong dự án Toyota?

Câu trả lời theo CV:

"Trong TMSS/TYT, tôi tham gia xây dựng các luồng đồng bộ sự kiện qua RabbitMQ. TMSS là hệ thống nghiệp vụ đại lý, còn TYT kết nối nhiều cổng và ứng dụng khách hàng, nên có những thay đổi cần truyền sang hệ thống khác mà không nên buộc mọi bước phụ hoàn tất ngay trong một request.

RabbitMQ giúp tách các bước xử lý đó và cho phép bên nhận xử lý theo tải của mình. Phần khó là dữ liệu có thể đã commit nhưng gửi thất bại, hoặc message được giao lại, nên tôi kết hợp với Inbox/Outbox và cơ chế xử lý lặp an toàn."

Ví dụ theo lĩnh vực trong CV: thay đổi liên quan đến khách hàng thân thiết ở một hệ thống cần được chuyển sang hệ thống khác. Tên sự kiện và điều kiện phát phải lấy đúng luồng bạn phụ trách; không tự gắn "hoàn tất sửa chữa luôn được cộng điểm" nếu nghiệp vụ thực tế có điều kiện khác.

Vì sao không gọi đồng bộ tất cả: nếu một bước không cần phản hồi tức thì, tách qua message có thể giảm phụ thuộc thời điểm. Nếu cần kết quả để quyết định ngay, vẫn có thể dùng REST/gRPC.

Đánh đổi cần nêu: dữ liệu phía nhận có thể cập nhật chậm, cần retry, giám sát và đối soát. Không nói "dùng RabbitMQ nên không mất dữ liệu".

Phân biệt dự án: CV nêu gRPC trong TYT. Điều đó không tự chứng minh mọi luồng TMSS đều dùng gRPC; mô tả từng hệ theo thành phần thực tế.

4.2. RabbitMQ khác Kafka thế nào?

Khía cạnh	RabbitMQ, theo mô hình queue phổ biến	Kafka
Trọng tâm	Phân phối message, định tuyến, giao việc	Nhật ký sự kiện phân vùng, đọc lại theo offset
Hoàn tất xử lý	ACK giúp broker biết delivery đã được nhận trách nhiệm	Consumer quản lý tiến độ qua offset
Nhiều nhóm nhận	Thường mỗi nhóm có queue riêng	Nhiều consumer group đọc cùng topic
Đọc lại	Cần thiết kế queue/lưu trữ; RabbitMQ Streams có mô hình khác	Là nhu cầu thiết kế quan trọng
Thứ tự	Phụ thuộc queue, consumer, retry và cách xử lý	Thứ tự trong từng partition

Ví dụ: gửi công việc tạo báo cáo hoặc thông báo thường phù hợp RabbitMQ. Lưu luồng hàng triệu sự kiện máy để nhiều hệ thống phân tích và đọc lại có thể phù hợp Kafka.

Không nên nói RabbitMQ không có khả năng lưu/đọc lại hoặc Kafka chỉ dùng dữ liệu lớn; sản phẩm có nhiều chế độ và cấu hình.

Chọn theo nhu cầu routing, lưu giữ, replay, throughput, độ trễ và năng lực vận hành. Kafka cũng không tự bảo đảm mọi side effect ở database ngoài chỉ xảy ra một lần. Nguồn: mô hình Kafka và partition.

4.3. ⭐ Transactional Outbox là gì? Vì sao cần?

Outbox giải quyết khoảng lỗi giữa commit nghiệp vụ và publish sự kiện, khi không có một giao dịch nguyên tử chung cho database và broker.

```
Một transaction SQL:
    cập nhật nghiệp vụ
    thêm sự kiện vào Outbox
    commit

Worker:
    nhận quyền gửi
    publish tới broker
    xác nhận kết quả theo cơ chế gửi
    đánh dấu đã gửi
```

Ví dụ: cập nhật membership thành công nhưng RabbitMQ mất kết nối. Nếu chỉ publish sau commit mà không lưu ý định gửi, sự kiện có thể bị bỏ quên. Với Outbox, bản ghi còn đó để worker thử lại.

Khoảng lỗi còn tồn tại: broker đã nhận nhưng worker chết trước khi đánh dấu. Worker sau có thể gửi lại cùng MessageId, nên consumer phải idempotent.

Liên hệ repo: LoyaltyTransactionalStore ghi aggregate và Outbox trong cùng callback transaction ở các phương thức tương ứng; LoyaltyOutboxPublisher nhận batch rồi gửi. Đây là bằng chứng cấu trúc Outbox, không phải bằng chứng mọi luồng đều đã được kiểm thử khi broker lỗi.

Outbox không thay thế cấu hình bền vững và publisher confirm. "Đã gọi hàm gửi" không nhất thiết có nghĩa broker đã nhận trách nhiệm lưu message.

Câu nói theo CV: "Trong TMSS, tôi tham gia triển khai đồng bộ RabbitMQ với Inbox/Outbox. Tôi dùng Outbox để lưu ý định gửi cùng giao dịch nghiệp vụ, sau đó worker xử lý gửi."

Khi trình bày code của TYT, có thể chỉ LoyaltyTransactionalStore và LoyaltyOutboxPublisher. Đừng dùng tên class của TYT để khẳng định đó là class trong TMSS; hai hệ liên quan nhưng khác codebase.

4.4. Transactional Inbox là gì?

Inbox lưu thông điệp nhận vào database để chống trùng và quản lý xử lý tin cậy. Có hai bố trí thường gặp:

1. Xử lý nghiệp vụ và đánh dấu message đã xử lý trong cùng transaction, sau đó ACK.
2. Lưu message bền vững trước, ACK sau khi lưu, rồi worker xử lý nghiệp vụ từ Inbox.

Với bố trí thứ hai, trách nhiệm retry đã chuyển từ broker sang bảng Inbox sau ACK. Worker phải bảo vệ việc xử lý lại và trạng thái hoàn tất.

Ví dụ: nhận sự kiện ERP tạo lệnh. Inbox lưu MessageId duy nhất và payload. Nếu broker giao lại, không tạo thêm một message nghiệp vụ tương đương. Worker kiểm tra rồi tạo lệnh một lần về mặt hiệu ứng.

Liên hệ source TYT: có ExternalRabbitMqConsumer lưu Inbox; LoyaltyInboxProcessor dispatch rồi đánh dấu processed. Chỉ riêng hai bước này không chứng minh cập nhật nghiệp vụ và đánh dấu processed nằm cùng transaction. Handler vẫn phải idempotent hoặc có ranh giới giao dịch phù hợp.

Đây là điểm cần nói rõ: có bảng tên Inbox không tự biến toàn hệ thống thành "đúng một lần".

4.5. ⭐ Bảo đảm idempotency khi nhận message thế nào?

Idempotency nghĩa là xử lý lặp cùng một yêu cầu nghiệp vụ không tạo thêm hiệu ứng ngoài ý định.

Thiết kế:

- Producer giữ MessageId ổn định qua retry.
- Consumer có khóa duy nhất theo phạm vi, ví dụ (ConsumerName, MessageId), hoặc khóa nghiệp vụ đủ chặt.
- Ghi dấu xử lý và cập nhật nghiệp vụ cùng transaction khi có thể.
- Có fingerprint để cùng mã nhưng nội dung khác được phát hiện.
- Tác dụng phụ tới dịch vụ ngoài dùng Outbox hoặc khóa idempotency phía dịch vụ đó.

Ví dụ: message "tiêu hao 2 kg cho lần thực hiện E1" đến ba lần. Chỉ một MaterialMovement được ghi nhờ unique key của lần thực hiện; tồn chỉ giảm 2 kg.

Hai cách không đủ: kiểm tra "chưa có" rồi INSERT bằng hai bước không nguyên tử; lưu MessageId vào Redis ngắn hạn rồi coi đó là bằng chứng vĩnh viễn.

Nếu thao tác là cộng/trừ, xử lý lặp nguy hiểm hơn cập nhật snapshot. Snapshot cũng cần kiểm tra phiên bản để bản cũ không ghi đè bản mới. Mục tiêu thực tế là hiệu ứng nghiệp vụ đúng một lần trong phạm vi thiết kế, trên nền delivery có thể lặp.

Điểm gắn hồ sơ: CV ghi bạn triển khai xử lý lặp an toàn trong đồng bộ TMSS. Hãy chuẩn bị đúng khóa duy nhất và ranh giới transaction ở consumer bạn phụ trách. Source TYT có script unique MessageKey cho Inbox, nhưng chưa chứng minh index đã tồn tại trên mọi database triển khai hoặc mọi tác dụng phụ đã nằm cùng transaction.

4.6. Database commit thành công nhưng publish RabbitMQ thất bại thì sao?

Nếu không có Outbox: có thể tồn tại dữ liệu nghiệp vụ nhưng hệ thống nhận không được thông báo. Không nên rollback giả bằng một UPDATE ngược tùy tiện vì có thể đã có tác động khác.

Nếu có Outbox: giao dịch đã lưu cả nghiệp vụ và message; worker giữ trạng thái chưa gửi, lưu lỗi và retry. API cần báo đúng trạng thái: nghiệp vụ đã chấp nhận, đồng bộ đang chờ, nếu hợp đồng có yêu cầu hiển thị.

Ví dụ: lệnh sản xuất đã hoàn thành trong MES nhưng ERP đang bảo trì. MES lưu sự kiện báo hoàn thành; dashboard tích hợp hiển thị "chờ đồng bộ". Khi ERP hồi phục, worker gửi lại cùng mã sự kiện.

Trường hợp khó: timeout publish không chứng minh broker chưa nhận. Vì vậy retry có thể sinh delivery lặp. Chỉ đánh dấu gửi thành công theo cơ chế xác nhận đã chọn; xử lý cả trường hợp không route được vào queue.

Đối soát phát hiện bản ghi chờ quá lâu, sự kiện bị lỗi cấu hình và dữ liệu không khớp. Không xóa Outbox pending chỉ để bảng gọn.

4.7. Xử lý lỗi, retry và dead-letter queue thế nào?

Phân loại trước khi retry:

Loại lỗi	Ví dụ	Cách xử lý
Tạm thời	Timeout, database mất kết nối ngắn	Retry có giới hạn, tăng thời gian chờ
Dữ liệu/hợp đồng	Payload sai, phiên bản không hỗ trợ	Cách ly, điều tra và sửa dữ liệu
Nghiệp vụ	Lệnh đã đóng, vật tư bị cấm	Ghi kết quả từ chối hoặc quy trình xử lý

Ví dụ lịch retry giả định: 5 giây, 20 giây, 1 phút, rồi chuyển trạng thái lỗi sau số lần cho phép. Thêm ngẫu nhiên để nhiều consumer không thử lại cùng lúc.

DLQ là queue đích cho message được dead-letter theo cấu hình; nó không tự xuất hiện chỉ vì ứng dụng có catch exception. Phải cấu hình exchange, binding và điều kiện chuyển, đồng thời giám sát.

Bẫy: NACK requeue ngay vô hạn gây vòng lặp nóng và chặn tải hữu ích. Khi replay sau sửa, giữ định danh phù hợp và vẫn kiểm tra idempotency.

Nếu ACK sau khi lưu Inbox, retry nghiệp vụ nằm ở Inbox; lỗi worker sau ACK không tự quay về RabbitMQ DLQ. Cần màn hình hoặc công cụ vận hành cho trạng thái lỗi đó.

4.8. ⭐ Ngăn nhiều worker cùng xử lý một dòng database queue thế nào?

Nhận quyền xử lý bằng một thao tác nguyên tử, không SELECT rồi UPDATE rời rạc. Ví dụ dưới đây giả định READ COMMITTED dùng khóa, RCSI tắt và schema hỗ trợ; phải điều chỉnh khi cấu hình khác.

```
;WITH nextJob AS
(
    SELECT TOP (1) *
    FROM dbo.JobQueue WITH (UPDLOCK, READPAST, ROWLOCK)
    WHERE (Status = 0 AND NextAttemptOnUtc ≤ @NowUtc)
       OR (Status = 1 AND LockedUntilUtc < @NowUtc)
    ORDER BY CreatedOnUtc, Id
)
UPDATE nextJob
SET Status = 1,
    LockToken = @LockToken,
    LockedUntilUtc = @LockedUntilUtc,
    AttemptCount = AttemptCount + 1
OUTPUT inserted.*;
```

Ví dụ: worker A nhận dòng 10; worker B nhận dòng khác hoặc không nhận được gì. Transaction nhận việc ngắn, không giữ khóa SQL trong suốt lời gọi ERP.

Sau khi nhận: cần lease, gia hạn cho tác vụ dài và điều kiện hoàn tất có LockToken. Nếu lease A hết và B nhận lại, A không được tự đánh dấu công việc của B.

Giới hạn: token khi đánh dấu hoàn tất chưa ngăn A thực hiện side effect muộn. Phải dùng idempotency/fencing tại nơi ghi nghiệp vụ hoặc cập nhật có điều kiện trong cùng transaction. Bảo đảm độc quyền mãi mãi không đến từ lease đơn thuần.

Trong TYT, procedure claim hiện có lease; không nên tự nhận source đã có đầy đủ token, heartbeat và fencing nếu chưa xác nhận.

Cách nói phù hợp CV: "Tôi đã làm cơ chế queue database trong luồng đồng bộ. Mục tiêu là nhận quyền xử lý bằng cập nhật nguyên tử và cho phép nhận lại khi worker ngừng." Sau đó giải thích lease và trạng thái đúng với implementation. Các cải tiến token/heartbeat/fencing ở ví dụ là điều cần đánh giá, không tự nhận đã có đầy đủ trong phiên bản bạn làm.

4.9. Duy trì nhất quán giữa các dịch vụ thế nào?

Tôi tách bất biến phải đúng ngay trong một chủ sở hữu dữ liệu, còn đồng bộ liên dịch vụ thường có độ trễ được kiểm soát.

Ví dụ: kiểm tra và trừ tồn do dịch vụ kho sở hữu. MES không tự cập nhật bảng kho từ bản cache, mà gửi yêu cầu cấp phát có định danh. Sau khi kho chấp nhận, MES chuyển sang trạng thái tiếp theo.

Các cơ chế thường dùng:

- Transaction cục bộ bảo vệ dữ liệu của từng dịch vụ.
- Outbox/Inbox và idempotency bảo vệ trao đổi.
- Phiên bản aggregate hoặc số thứ tự để phát hiện message cũ.
- Quy trình nhiều bước có trạng thái, timeout và hành động bù khi cần.
- Đối soát định kỳ và cơ chế sửa có audit.

Ví dụ bù: giữ chỗ vật tư thành công nhưng lệnh bị hủy trước sản xuất; phát yêu cầu giải phóng giữ chỗ. Nếu đã tiêu hao vật lý, không thể chỉ "rollback" như chưa từng xảy ra; cần quy trình hoàn trả/điều chỉnh được chấp thuận.

Không dùng một transaction SQL đang mở để chờ chuỗi HTTP qua nhiều dịch vụ.

4.10. ⭐ Bạn xử lý đổi voucher có nhiều yêu cầu đồng thời thế nào?

Câu trả lời gắn với công việc Loyalty/voucher đã ghi trong CV:

"Trong TYT, tôi tham gia xây dựng luồng tích lũy, đổi điểm và phát hành/đổi voucher. Với bài toán nhiều yêu cầu đồng thời, tôi tách rõ ba việc phải bảo vệ: điểm không bị trừ vượt, hạn mức voucher không bị dùng vượt và cùng một yêu cầu không tạo hai lần đổi.

Điểm tôi chú ý là kiểm tra bằng SELECT hoặc chặn nút trên giao diện không đủ khi có nhiều request hoặc nhiều instance. Quyết định cuối cùng phải được bảo vệ ở nơi lưu dữ liệu, còn những bước đồng bộ ra hệ thống khác cần có định danh và cơ chế xử lý lại."

Ví dụ thiết kế để giải thích: khách có 1.000 điểm, mỗi voucher cần 800 điểm, chỉ còn một suất. Hai request cùng tới thì chỉ được một lần đổi thành công. Kiểm tra đủ điểm và đủ suất phải gắn với cập nhật nguyên tử trong phạm vi giao dịch phù hợp.

Nếu hai dữ liệu cùng database, có thể dùng UPDATE có điều kiện, unique key theo yêu cầu, ghi lịch sử và Outbox trong cùng đơn vị giao dịch. Nếu thuộc hai dịch vụ riêng, cần giữ chỗ/xác nhận/hủy hoặc một quy trình phối hợp tương ứng; không nhận một transaction SQL bao phủ được toàn mạng.

Về hết hạn điểm: CV nêu kiểm soát hết hạn. Khi bị hỏi sâu, giải thích theo mô hình thật: điểm theo đợt nào, hết hạn lúc nào, ưu tiên trừ phần nào, cách hoàn lại khi hủy và múi giờ áp dụng. Không tự nói đã dùng một thuật toán cụ thể nếu chưa đối chiếu implementation.

Phạm vi bằng chứng: CV xác nhận bạn khai kinh nghiệm nghiệp vụ này; ví dụ SQL và các cơ chế trên là cách giải thích thiết kế. Không tự nhận mọi cơ chế đều đã có ở mọi đường code. Source TYT có lưu aggregate/lịch sử/Outbox qua transaction, nhưng chưa đủ kết luận toàn bộ đường trừ điểm và hạn mức đã được kiểm chứng đồng thời.

4.11. REST/gRPC đồng bộ khác messaging bất đồng bộ thế nào?

Tiêu chí	REST/gRPC theo request–response	Message bất đồng bộ
Người gọi	Chờ phản hồi theo timeout	Thường không chờ xử lý nghiệp vụ cuối
Phụ thuộc thời điểm	Bên nhận cần đáp ứng lúc gọi	Bên nhận có thể xử lý sau
Báo lỗi	Phản hồi hoặc timeout	Trạng thái xử lý, retry và kênh kết quả
Độ phức tạp chính	Timeout, retry, chuỗi phụ thuộc	Trùng, thứ tự, backlog và đối soát

Ví dụ: kiểm tra một số serial đang thuộc lệnh nào có thể cần request–response. Phát sự kiện công đoạn hoàn tất cho hệ thống báo cáo thường phù hợp bất đồng bộ.

Đồng bộ không có nghĩa chỉ chạy một thread; HTTP/gRPC vẫn có thể dùng I/O async. Bất đồng bộ không có nghĩa chắc chắn không mất dữ liệu.

Shared và gRPC: Shared là hợp đồng; gRPC là cơ chế gọi implementation trên process khác. Nếu gọi implementation qua DI trong cùng process thì tránh được truyền mạng, nhưng đó là quyết định đưa nghiệp vụ vào cùng ứng dụng.

Với Protobuf, mã field phải ổn định và không trùng trong phạm vi tương ứng, gồm tag kế thừa. Đây là chi phí quản lý schema. Nguồn: quy tắc định danh protobuf-net.

4.12. Đồng bộ dữ liệu sản xuất MES–ERP đáng tin cậy thế nào?

Xác định chủ sở hữu từng dữ liệu trước: sản phẩm/BOM/kế hoạch thường được quản trị tại hệ thống đã thống nhất; MES ghi thực thi, ERP nhận kết quả phục vụ kế hoạch hoặc hạch toán. Đây là thiết kế cần thỏa thuận, không phải quy tắc cứng mọi nhà máy.

Luồng minh họa:

1. ERP phát hành lệnh và phiên bản BOM/routing.
2. MES nhận qua Inbox, kiểm tra và phản hồi trạng thái chấp nhận/từ chối.
3. MES ghi kết quả, tiêu hao và Outbox cùng transaction phù hợp.
4. Worker gửi sự kiện có EventId, mã lệnh, phiên bản, nhà máy và đơn vị tính.
5. ERP xử lý idempotent và phản hồi kết quả nghiệp vụ.
6. Đối soát phát hiện thiếu, trùng hoặc chênh lệch số lượng.

Ví dụ: MES báo đạt 95, lỗi 5. ERP timeout sau khi ghi nhận; retry cùng EventId phải trả kết quả đã xử lý, không nhập kho thêm 95 lần nữa.

Không xem HTTP 200 hoặc publisher confirm là bằng chứng ERP đã hạch toán thành công nếu nó mới chỉ nhận message. Phân biệt "đã gửi", "đã nhận" và "đã áp dụng nghiệp vụ". Nguồn: phạm vi xác nhận RabbitMQ.

---

5. MES / ERP / nghiệp vụ sản xuất

5.1. ⭐ MES là gì và khác ERP thế nào?

MES là hệ thống quản lý thực thi sản xuất: theo dõi lệnh được thực hiện ra sao, ở công đoạn nào, bằng máy/người/vật tư nào, với sản lượng và chất lượng thực tế thế nào.

ERP quản trị nguồn lực doanh nghiệp ở phạm vi rộng hơn: kế hoạch, mua hàng, bán hàng, tồn kho, tài chính và các nghiệp vụ khác tùy sản phẩm.

Ví dụ: ERP xác định tuần này cần sản xuất 1.000 bộ linh kiện. MES phân phối công việc xuống trạm, ghi bắt đầu/kết thúc, lô vật tư, 950 bộ đạt và 50 bộ cần xử lý. Kết quả được gửi lại ERP theo quy trình đã thống nhất.

Ranh giới có thể giao nhau; không phải mọi nhà máy đều dùng cùng cách chia chức năng. Phải xác định hệ thống nào sở hữu lệnh, tồn, BOM và số liệu hoàn thành.

Cách nói khi phỏng vấn: "ERP hỗ trợ quyết định cần làm gì và quản lý tài nguyên doanh nghiệp; MES tập trung kiểm soát và ghi nhận việc đang thực hiện tại xưởng." Không nên nói ERP hoàn toàn không theo dõi sản xuất. Nguồn: tổng quan MES của Siemens.

5.2. WMS là gì và liên quan MES thế nào?

WMS quản lý hoạt động kho: nhận hàng, vị trí chứa, cất hàng, lấy hàng, di chuyển, kiểm đếm và xuất hàng. MES quản lý vật tư được sử dụng trong quá trình sản xuất.

Ví dụ phối hợp:

1. MES cần 100 kg nhựa cho lệnh SX-01.
2. Hệ thống kho xác định lô phù hợp, trạng thái chất lượng và vị trí.
3. WMS hướng dẫn lấy 100 kg và chuyển đến khu cấp liệu.
4. MES ghi lượng thực tế đã dùng, còn lại, hao hụt và trả lại.

Phân biệt: xuất vật tư khỏi vị trí kho không tự đồng nghĩa đã tiêu hao để tạo sản phẩm. Vật tư có thể nằm tại xưởng chờ dùng.

Thiết kế phải làm rõ tồn kho vật lý, tồn tại xưởng, giữ chỗ và tiêu hao thuộc hệ thống nào. Nếu WMS và MES cùng trừ cho một sự kiện thì số liệu sẽ lệch.

Đồng bộ dùng mã giao dịch, lô, vị trí, đơn vị tính và phản hồi xử lý. Khi chuyển thất bại, trạng thái phải thể hiện đang chờ hoặc bị từ chối thay vì báo thành công ở cả hai bên theo suy đoán.

5.3. ⭐ BOM là gì?

BOM là danh mục cấu thành sản phẩm: cần những vật tư/bán thành phẩm nào và lượng bao nhiêu để tạo một lượng sản phẩm theo cơ sở định mức xác định.

Ví dụ BOM cho một bộ bàn:

Thành phần	Định mức
Mặt bàn	1 tấm
Chân bàn	4 chiếc

Thành phần	Định mức
Vít	16 chiếc

Sản xuất 10 bộ cần lý thuyết 10 mặt, 40 chân và 160 vít, chưa tính quy tắc hao hụt hoặc thay thế.

BOM cần phiên bản, thời gian hiệu lực, đơn vị tính, lượng cơ sở, cấu trúc nhiều cấp và quy tắc phê duyệt. BOM của một bộ khác BOM của 100 kg sản phẩm; phải tính theo đúng cơ sở.

Ví dụ hao hụt: nếu quy tắc là cộng thêm 2% trên định mức, 100 kg thành 102 kg. Nếu quy tắc là hiệu suất thu hồi 98%, đầu vào cho 100 kg đầu ra là 100 / 0,98. Hai công thức khác nhau, không được dùng thay thế tùy tiện.

Khi lệnh được phát hành, ghi phiên bản hoặc snapshot BOM để lịch sử không bị thay đổi khi danh mục gốc được cập nhật. Nguồn: BOM và công thức sản xuất.

5.4. ⭐ Routing trong sản xuất là gì?

Routing mô tả quy trình thực hiện: những công đoạn nào, thứ tự/quan hệ phụ thuộc, nguồn lực, thời gian và điều kiện thực hiện.

Ví dụ:

```
Cắt vật liệu → Gia công → Kiểm tra kích thước → Lắp ráp → Kiểm tra cuối
```

Mỗi công đoạn có mã, trạm hoặc nhóm máy, hướng dẫn, thời gian chuẩn, dữ liệu phải ghi và điều kiện chuyển bước. Quy trình thực tế có thể có nhánh song song, gia công ngoài hoặc vòng sửa lại, không chỉ là một cột số thứ tự.

Phân biệt: BOM trả lời cần gì; routing trả lời làm như thế nào và ở đâu.

Ví dụ kiểm soát: sản phẩm chỉ được vào công đoạn lắp ráp khi kiểm tra kích thước đã đạt. Nếu không đạt, chuyển nhánh sửa lại thay vì bỏ qua.

Khi thiết kế, lưu phiên bản routing theo lệnh; lưu từng lần thực hiện để phân biệt chạy lần đầu và chạy lại. Không sửa trạng thái công đoạn cũ thành chưa chạy rồi xóa dấu vết lần lỗi. Nguồn: routing và công đoạn.

5.5. Lệnh sản xuất là gì?

Lệnh sản xuất là đối tượng giao việc tạo một sản phẩm hoặc bán thành phẩm, với số lượng, thời gian và quy trình đã xác định.

Thông tin thường có: mã lệnh, sản phẩm, lượng kế hoạch, đơn vị tính, nhà máy, ngày bắt đầu/kết thúc dự kiến, BOM/routing đã chọn và trạng thái.

Ví dụ: SX-2026-001 yêu cầu làm 500 bộ linh kiện tại dây chuyền A, theo BOM phiên bản 3 và routing phiên bản 2.

Vòng đời minh họa: Nháp → Phát hành → Đang thực hiện → Hoàn tất → Đóng. Tạm dừng, hủy và mở lại phải có quy tắc và quyền riêng.

Phân biệt hoàn tất và đóng: hoàn tất có thể là đã báo sản lượng; đóng có thể bao gồm đối soát vật tư và xử lý chênh lệch. Ý nghĩa phải thống nhất theo nhà máy.

Trước phát hành có thể cần kiểm tra dữ liệu kỹ thuật và kế hoạch. Trước đóng cần kiểm tra lượng tốt/lỗi, vật tư đã ghi nhận và các vấn đề chất lượng chưa xử lý. Nguồn: vòng đời lệnh sản xuất.

5.6. ⭐ WIP là gì?

WIP là sản phẩm hoặc vật tư đang trong quá trình sản xuất, chưa trở thành thành phẩm hoàn tất theo điểm kiểm soát đã định.

Ví dụ: một lệnh 100 bộ có 40 bộ đang gia công, 30 bộ chờ lắp ráp và 20 bộ đang kiểm tra cuối; 10 bộ đã hoàn tất và xuất khỏi phạm vi WIP. Theo cách phân loại này, WIP là 90 bộ.

WIP có thể gồm đang chạy, chờ, đang vận chuyển, tạm giữ chất lượng hoặc sửa lại. Cần định nghĩa ranh giới đo; giá trị kế toán WIP cũng không đồng nhất hoàn toàn với số lượng hiện trường.

Ứng dụng: nếu 200 chi tiết chờ trước một máy trong khi công đoạn sau rảnh, đó là dấu hiệu cần điều tra nút thắt, lịch chạy hoặc chất lượng.

Lưu ý tính toán: không cộng sản lượng tích lũy của tất cả công đoạn để ra WIP. Cùng một sản phẩm đi qua ba công đoạn sẽ bị đếm nhiều lần. Cần trạng thái/vị trí hiện tại hoặc sổ chuyển WIP đủ để đối chiếu đầu vào, đầu ra và hao hụt.

5.7. LOT khác số serial như thế nào?

LOT/lô nhận diện một nhóm vật tư hoặc sản phẩm có chung nguồn gốc hay điều kiện sản xuất theo quy tắc doanh nghiệp. **Serial** nhận diện từng đơn vị riêng.

Ví dụ: lô pin PIN-01 chứa 1.000 viên; từng viên có serial riêng nếu cần quản lý ở cấp đơn vị. Một cuộn thép có thể quản lý theo lô/cuộn và lượng, không nhất thiết từng gram có serial.

Nhu cầu	Theo lô	Theo serial
Nhận diện	Một nhóm	Một đơn vị
Số lượng	Có thể lớn hoặc phân chia	Thường một đơn vị định danh
Truy vết	Phạm vi nhóm	Chi tiết từng sản phẩm

Serial không nhất thiết duy nhất toàn cầu; unique key phải theo quy ước như tenant, sản phẩm và serial. Không giả định chỉ riêng chuỗi barcode luôn đủ định danh.

Khi lô bị chia hoặc trộn, cần lưu quan hệ nguồn–đích và lượng. Một serial thành phẩm có thể chứa nhiều linh kiện theo lô hoặc theo serial, nên mô hình truy xuất phải hỗ trợ cả hai.

5.8. ⭐ Traceability là gì và vì sao quan trọng?

Traceability là khả năng lần theo nguồn gốc và quá trình của vật tư/sản phẩm.

- Truy ngược: thành phẩm này dùng lô nguyên liệu nào, sản xuất bằng máy nào, ai thực hiện và kiểm tra ra sao?
- Truy xuôi: một lô nguyên liệu lỗi đã đi vào những lệnh, lô thành phẩm và đơn vị giao hàng nào?

Ví dụ: nhà cung cấp thông báo lô linh kiện L-100 bị lỗi. Hệ thống xác định thành phẩm có liên quan để cách ly hoặc thu hồi đúng phạm vi, thay vì dừng toàn bộ sản phẩm.

Cần mã định danh, liên kết đầu vào–đầu ra, lượng, công đoạn, thời gian, máy/người và phiên bản dữ liệu kỹ thuật. Quét barcode là một cách thu thập, chưa tự tạo ra traceability nếu không lưu các liên kết này.

Giới hạn độ chính xác: nếu vật tư đã trộn nhưng chỉ lưu theo lệnh, không thể tuyên bố biết chính xác từng serial dùng phần nào. Kết quả truy xuất phải thể hiện phạm vi thực sự có dữ liệu.

Traceability hỗ trợ chất lượng, điều tra nguyên nhân và giảm phạm vi ảnh hưởng. Nguồn: traceability trong MES.

Cầu nối từ CV: lịch sử cập nhật sản phẩm trong OESMedicalAPI và lịch sử điểm/voucher trong TYT giúp bạn nói về audit và đối soát. Khi chuyển sang MES, phải bổ sung quan hệ vật tư–công đoạn–đầu ra. Không gọi lịch sử điểm hay lịch sử sửa danh mục là kinh nghiệm traceability sản xuất trực tiếp.

5.9. Kế hoạch sản xuất khác thực thi sản xuất thế nào?

Kế hoạch xác định sản xuất gì, bao nhiêu, khi nào và dự kiến dùng nguồn lực nào. Thực thi ghi nhận và điều khiển việc thực tế xảy ra tại xưởng.

Ví dụ: kế hoạch giao dây chuyền A làm 500 bộ trong ca sáng. Thực tế máy dừng 30 phút, làm được 420 bộ đạt và 15 bộ lỗi. MES ghi nhận thực tế và gửi chênh lệch để bộ phận kế hoạch điều chỉnh.

Kế hoạch cần nhu cầu, năng lực, thời gian chuẩn và tồn khả dụng dự kiến. Thực thi cần trạng thái máy, thao tác người vận hành, vật tư thực dùng và kết quả chất lượng.

Thiết kế: không ghi đè PlannedQuantity bằng ActualQuantity vì muốn hai con số khớp nhau. Giữ riêng kế hoạch và thực tế, lịch sử thay đổi và lý do.

Khi điều chỉnh kế hoạch cho lệnh đang chạy, phải xét vật tư đã cấp, công đoạn đã hoàn tất và dữ liệu đã gửi ERP. Đó là thay đổi nghiệp vụ có kiểm soát, không chỉ sửa một trường số lượng.

5.10. Tiêu hao vật tư trong MES hoạt động thế nào?

Tiêu hao là ghi nhận vật tư thực sự được dùng cho sản xuất, theo lệnh/công đoạn, lô, lượng và đơn vị.

Hai cách ghi nhận phổ biến:

- Ghi thực tế: người vận hành quét/cân hoặc thiết bị gửi số lượng.
- Tính theo định mức khi xác nhận sản lượng, thường gọi là backflush.

Ví dụ: định mức 2 vít cho một sản phẩm; báo 100 sản phẩm thì hệ thống dự tính tiêu hao 200 vít. Nếu thực tế dùng 205 vì lỗi, cần chính sách ghi phần chênh lệch, không âm thầm sửa định mức.

Luồng thường gồm giữ chỗ → cấp tới xưởng → tiêu hao → trả dư hoặc điều chỉnh. Không phải mỗi nhà máy đều áp dụng đủ bốn bước.

Kiểm soát: vật tư đúng BOM hoặc được phép thay thế, lô được phép dùng, đủ tồn, đúng đơn vị, không ghi trùng và không vượt mức được duyệt. Khi có giữ chỗ, tiêu hao phải giảm đồng thời tồn và phần giữ chỗ theo mô hình; không trừ khả dụng hai lần.

Nếu đã báo tiêu hao sai, tạo bản điều chỉnh liên kết giao dịch gốc, có quyền và lý do.

5.11. Theo dõi trạng thái từng công đoạn thế nào?

Thiết kế một mô hình trạng thái rõ ràng cho từng lần thực hiện:

```
Chưa sẵn sàng → Sẵn sàng → Đang chạy → Hoàn tất
                              ↕
                           Tạm dừng

Chất lượng không đạt → Tạm giữ → Sửa lại hoặc Loại bỏ
```

Đây là mô hình minh họa; cần tách trạng thái chất lượng khỏi trạng thái thực hiện nếu một trường Status không biểu đạt đủ.

Ví dụ: công đoạn OP20 chỉ sẵn sàng khi OP10 hoàn tất và kết quả bắt buộc đạt. Người vận hành bắt đầu OP20 thì ghi máy, ca, người và thời gian. Khi kết thúc, ghi lượng đạt/lỗi cùng dữ liệu đo.

Chuyển trạng thái phải kiểm tra ở backend, kèm phiên bản để hai trạm không cùng bắt đầu trái phép. Mỗi lần sửa lại có ExecutionId mới, liên kết lần gốc.

Lưu cả trạng thái hiện tại để đọc nhanh và sự kiện chuyển trạng thái để điều tra. Một thao tác cập nhật trạng thái, lịch sử và Outbox cần nguyên tử trong phạm vi database phù hợp.

5.12. ⭐ Thiết kế quy trình tích hợp ERP–MES thế nào?

Bước đầu là hợp đồng nghiệp vụ, gồm nguồn dữ liệu chính, mã định danh, đơn vị, phiên bản và ý nghĩa trạng thái.

Chiều minh họa	Dữ liệu
ERP → MES	Sản phẩm, lệnh, BOM/routing được duyệt
WMS/ERP → MES	Cấp phát, trạng thái lô, khả dụng theo thỏa thuận

Chiều minh họa	Dữ liệu
MES → ERP	Kết quả sản xuất, tiêu hao, phế phẩm, trạng thái hoàn tất

Ví dụ toàn luồng: ERP phát hành lệnh SX-01 với phiên bản 3. MES lưu Inbox và kiểm tra dữ liệu tham chiếu. Nếu thiếu vật tư danh mục, ghi lỗi chờ xử lý; không tự tạo một mã mới sai chuẩn. Khi chấp nhận, MES thực thi theo phiên bản 3.

Khi hoàn thành, MES lưu kết quả và Outbox. ERP nhận sự kiện có khóa ổn định rồi ghi nhận theo nghiệp vụ. MES theo dõi phản hồi đã áp dụng, không chỉ phản hồi truyền nhận.

Trường hợp thay đổi: ERP gửi hủy trong khi công đoạn đã chạy. MES phản hồi điều kiện không cho hủy trực tiếp hoặc chuyển quy trình xử lý, không xóa lịch sử.

Cần retry, idempotency, phát hiện phiên bản cũ, đối soát và màn hình vận hành. Transport có thể là API, message hoặc phương thức hệ thống hỗ trợ.

5.13. Xử lý sản phẩm lỗi và kết quả kiểm tra chất lượng thế nào?

Lưu kế hoạch kiểm tra, chỉ tiêu, kết quả và quyết định xử lý. Một bản ghi thường gồm lệnh/công đoạn, lô/serial, đặc tính đo, giá trị, đơn vị, giới hạn, người/máy và thời điểm.

Ví dụ: đường kính cho phép 10,00 ± 0,05 mm; đo 10,12 mm là ngoài giới hạn. Sản phẩm chuyển trạng thái tạm giữ, chưa được dùng như hàng đạt cho bước sau.

Quyết định xử lý có thể là sửa lại, loại bỏ, kiểm tra lại hoặc chấp nhận có phê duyệt. Người đo và người được quyền quyết định không nhất thiết là một.

Dữ liệu quan trọng: lưu phiên bản tiêu chuẩn đã dùng. Nếu tiêu chuẩn đổi sau một tháng, kết quả cũ vẫn phải được đánh giá theo quy tắc tại thời điểm kiểm tra.

Khi sửa lại, tạo lần thực hiện và kiểm tra mới, giữ kết quả lỗi ban đầu. Đồng bộ đúng ảnh hưởng tới tồn, WIP và ERP. Một nhãn "đã sửa" không đủ chứng minh sản phẩm được kiểm lại và đạt yêu cầu.

5.14. Xây dashboard tiến độ sản xuất gần thời gian thực thế nào?

Tách đường ghi nghiệp vụ khỏi mô hình đọc phục vụ dashboard:

```
Kết quả công đoạn → Database + Outbox
                          ↓
                   Xử lý cập nhật
                          ↓
              Mô hình đọc → API → Giao diện
                                    ↑
                       SignalR/thông báo thay đổi
```

Ví dụ: dashboard hiển thị kế hoạch 1.000, đạt 650, lỗi 20, số đang làm và thời điểm cập nhật cuối. Không nhất thiết mỗi sự kiện phải làm trình duyệt tải lại toàn bộ ngày sản xuất.

Có thể đẩy bản cập nhật theo nhà máy/dây chuyền được phân quyền hoặc thông báo để client tải lại phần bị ảnh hưởng. Khi mất kết nối, tải snapshot và bắt kịp theo phiên bản/cursor.

Độ đúng: xác định "đạt" được tính ở công đoạn cuối hay mọi công đoạn; nếu cộng tất cả, cùng sản phẩm bị đếm nhiều lần. Phân biệt dữ liệu tạm tính và đã chốt.

Với chỉ số như OEE, phải thống nhất thời gian kế hoạch, thời gian chạy, tốc độ chuẩn và quy tắc chất lượng. Đồ thị đẹp nhưng công thức sai không giúp điều hành.

5.15. ⭐ Ngăn lệnh tiêu hao vượt vật tư khả dụng thế nào?

Bảo vệ bằng database tại hệ thống sở hữu tồn, theo đúng khóa tenant, kho, vị trí, vật tư và lô cần thiết.

Ví dụ: tồn vật lý 100 kg, trong đó 20 kg bị cách ly và 30 kg đã giữ cho lệnh khác. Theo mô hình giả định các nhóm không chồng nhau, lượng tự do là 50 kg; lệnh mới không được lấy 60 kg.

Nếu có giữ chỗ: giữ 40 kg cho lệnh SX-01 bằng cập nhật nguyên tử. Khi SX-01 tiêu 10 kg, giảm tồn vật lý và giữ chỗ của chính lệnh; không kiểm tra như đang yêu cầu 10 kg mới từ lượng tự do của người khác.

Nếu không giữ chỗ: dùng UPDATE có điều kiện hoặc giao dịch phân bổ thích hợp, kết hợp khóa chống trùng như mục 3.12.

Ngoài số lượng, kiểm tra chất lượng lô, hạn dùng, đơn vị và quyền dùng vật tư thay thế. Tiêu hao vượt định mức là một quy tắc khác với vượt tồn: đủ tồn vẫn có thể cần duyệt nếu vượt định mức.

Không dựa trên số tồn đang hiện trên frontend hoặc cache để quyết định cuối cùng.

---

6. PLC / máy quét / mã vạch / tích hợp thiết bị

6.1. ⭐ PLC là gì và có vai trò gì trong sản xuất?

PLC là bộ điều khiển lập trình được, dùng để đọc tín hiệu đầu vào và điều khiển đầu ra theo chương trình. Nó thường thực hiện logic máy, liên động và các hoạt động cần phản ứng ở cấp thiết bị.

Ví dụ: cảm biến báo có phôi; PLC điều khiển cơ cấu kẹp và chu trình gia công; khi hoàn tất, PLC cập nhật bộ đếm hoặc phát tín hiệu trạng thái.

MES nhận kết quả, liên kết với lệnh/serial, quản lý lịch sử và điều phối nghiệp vụ. MES không nên được mặc định là nơi thay thế vòng điều khiển thời gian thực hoặc cơ chế an toàn của máy.

Tích hợp: tùy thiết bị, dùng OPC UA, driver của hãng, giao thức công nghiệp, gateway hoặc TCP theo đặc tả. "Thiết bị có cổng mạng" chưa chứng minh có thể đọc bằng một socket tùy ý.

Trả lời nếu chưa có kinh nghiệm: "Tôi hiểu vai trò và cách tổ chức tích hợp; tôi chưa trực tiếp lập trình PLC trong nhà máy. Tôi sẽ phối hợp kỹ sư tự động hóa để xác nhận giao thức và quy tắc điều khiển."

Điều chỉnh theo hồ sơ: CV nêu tích hợp phần mềm doanh nghiệp, không nêu trực tiếp lập trình PLC hay driver thiết bị. Giữ phần này là kiến thức và cách tiếp cận; chỉ bổ sung một hãng/giao thức đã dùng nếu bạn có trải nghiệm ngoài CV.

6.2. TCP/IP, MQTT và OPC UA khác nhau thế nào?

Ba khái niệm không ở cùng một cấp:

Công nghệ	Vai trò	Việc ứng dụng còn phải xác định
TCP/IP	Nền tảng truyền dữ liệu mạng; TCP cung cấp luồng byte tin cậy có thứ tự trong kết nối	Định dạng message, phân khung, ý nghĩa dữ liệu
MQTT	Giao thức publish/subscribe qua broker	Topic, schema, quyền, độ bền và xử lý nghiệp vụ
OPC UA	Kiến trúc/giao thức trao đổi dữ liệu công nghiệp với mô hình thông tin	Node, quyền truy cập, chứng chỉ, mapping nghiệp vụ

Ví dụ: thiết bị cũ gửi frame riêng qua TCP; gateway chuyển thành message MQTT; MES đọc dữ liệu máy qua OPC UA nếu thiết bị hỗ trợ.

OPC UA có đọc/ghi, subscription, sự kiện và mô hình dữ liệu; không chỉ là "TCP có thêm tên biến". Nguồn: OPC Foundation.

MQTT có các mức QoS, nhưng bảo đảm giao thức không tự bao phủ transaction trừ tồn trong SQL. Vẫn phải chống trùng ở nghiệp vụ. Nguồn: đặc tả MQTT.

6.3. ⭐ Tích hợp máy quét mã vạch với ứng dụng .NET thế nào?

Trước hết xác định chế độ thiết bị:

- Giả lập bàn phím: máy quét nhập chuỗi và ký tự kết thúc vào ô đang chọn.
- Cổng serial/USB COM: ứng dụng nền hoặc desktop đọc theo cấu hình.
- TCP hoặc SDK của hãng: kết nối theo đặc tả.
- Camera trên web/di động: giải mã qua thư viện phù hợp.

Ví dụ giao diện web: công nhân quét serial tại trạm lắp ráp. Giao diện thu chuỗi tới ký tự kết thúc, gọi API kèm StationId, ExecutionId và RequestId. Backend kiểm tra sản phẩm, công đoạn, quyền trạm và trạng thái hiện tại.

Các chi tiết dễ lỗi: mất focus, ký tự cuối CR/LF, định dạng GS1 có ký tự phân cách, khác layout bàn phím, quét thiếu và hai trạm cùng nhận một serial.

Trình duyệt thông thường không tùy ý mở kết nối TCP tới mọi máy quét; có thể cần agent/gateway tại xưởng hoặc chế độ bàn phím.

Không chỉ kiểm tra chuỗi có đúng độ dài. Barcode hợp lệ về hình thức vẫn có thể thuộc sản phẩm khác, đã bị hủy hoặc đang ở công đoạn khác.

6.4. Giao tiếp thiết bị sản xuất qua TCP socket thế nào?

Đọc tài liệu giao thức: ai kết nối trước, port, encoding, định dạng frame, độ dài, checksum, mã lệnh và ACK. TCP là luồng byte; một lần Read không tương ứng một message.

Ví dụ giả định giao thức dùng 4 byte độ dài big-endian:

```
var header = new byte[4];
await stream.ReadExactlyAsync(header, ct);
var length = BinaryPrimitives.ReadInt32BigEndian(header);

if (length ≤ 0 || length > 64 * 1024)
    throw new InvalidDataException("Độ dài bản tin không hợp lệ.");

var payload = new byte[length];
await stream.ReadExactlyAsync(payload, ct);
await PersistDeviceMessageAsync(payload, ct);
```

Chỉ dùng cách này nếu thiết bị thực sự có định dạng trên. Với delimiter hoặc STX/ETX phải có parser tương ứng.

Vận hành: timeout cho kết nối/đọc, heartbeat nếu giao thức yêu cầu, reconnect có backoff, giới hạn buffer và xử lý kết nối đóng giữa frame. Một frame có thể bị chia qua nhiều gói; nhiều frame có thể tới cùng lúc.

ACK ứng dụng phải có ý nghĩa rõ: nhận byte, lưu bền vững hay hoàn tất nghiệp vụ. Nếu cần không mất dữ liệu sau ACK, không gửi ACK trước điểm lưu bền vững tương ứng.

6.5. WebSocket khác HTTP thế nào?

HTTP theo mô hình phổ biến là client gửi request và server phản hồi. WebSocket cung cấp kênh hai chiều lâu dài sau bắt tay, phù hợp cập nhật liên tục.

Ví dụ: HTTP tải danh sách lệnh và gửi thao tác xác nhận; WebSocket hoặc SignalR đẩy thay đổi tiến độ tới dashboard. SignalR có thêm abstraction và có thể dùng transport khác theo môi trường.

Không cần mở WebSocket chỉ để tạo một lệnh. Ngược lại, polling HTTP quá dày cho hàng nghìn màn hình có thể tạo nhiều request không có dữ liệu mới.

Điểm cần thiết kế: xác thực, quyền theo nhóm nhà máy, reconnect, heartbeat, giới hạn tốc độ, xử lý client chậm và làm mới snapshot sau mất kết nối.

WebSocket không tự là một hàng đợi bền vững. Client mất mạng có thể bỏ lỡ bản cập nhật; cần phiên bản/cursor hoặc tải lại dữ liệu. HTTP cũng có thể stream hoặc dùng SSE, nên không mô tả HTTP là tuyệt đối không hỗ trợ cập nhật liên tục.

6.6. Xử lý quét barcode trùng thế nào?

Phân biệt ba trường hợp:

1. Thiết bị gửi lại cùng lần quét do chưa nhận ACK.
2. Người dùng quét lại sản phẩm tại cùng bước.
3. Cùng mã được quét hợp lệ tại một bước/lần thực hiện khác.

Thiết kế: dùng ScanEventId hoặc mã yêu cầu ổn định cho lần truyền lại. Unique key nghiệp vụ có thể gồm tenant, serial, công đoạn và lần thực hiện, tùy quy tắc. Kiểm tra và ghi phải nguyên tử.

Ví dụ: serial S001 tại OP20 chỉ được xác nhận hoàn tất một lần trong ExecutionId E1. Request lặp E1 trả kết quả đã ghi. S001 vào OP30 vẫn phải được phép vì là nghiệp vụ khác.

Debounce ở frontend giảm thao tác lặp, nhưng không bảo vệ giữa hai trạm hoặc khi server chạy nhiều instance. Dùng timestamp trong vài giây làm khóa duy nhất cũng có thể loại nhầm lần quét hợp lệ.

Nếu cùng ScanEventId nhưng payload khác, báo xung đột để điều tra; không âm thầm trả thành công.

6.7. ⭐ PLC gửi dữ liệu khi MES mất kết nối thì xử lý thế nào?

Thiết kế lưu đệm bền vững gần thiết bị, thường tại gateway/edge nếu PLC không có khả năng lưu đủ.

```
PLC → Gateway lưu bền vững → Gửi MES → MES commit → Xác nhận
              ↑                              |
              └──── giữ dữ liệu khi chưa được xác nhận
```

Ví dụ: gateway nhận 500 kết quả trong lúc MES ngừng 10 phút. Mỗi sự kiện có DeviceId, phiên khởi động, sequence, thời gian và payload. Khi kết nối hồi phục, gửi lại; MES dùng khóa ổn định chống trùng.

Gateway chỉ xác nhận theo khả năng giao thức và điểm lưu bền vững đã thống nhất. Bộ nhớ RAM thuần có thể mất khi gateway tắt nguồn. Cần dung lượng, thời gian lưu tối đa và cảnh báo gần đầy.

Giới hạn quan trọng: PLC không lưu, không chờ ACK và gateway cũng ngừng thì không thể hứa khôi phục dữ liệu chưa từng được ghi ở đâu. Phải đánh giá từ đầu.

Việc máy tiếp tục chạy hay dừng theo chế độ suy giảm phải do nhà máy và kỹ sư điều khiển phê duyệt theo quy trình, không do developer tùy ý quyết định.

Điểm nối phù hợp với bạn: kinh nghiệm Inbox/Outbox, worker và dữ liệu cần xử lý lại hỗ trợ tư duy lưu đệm tại gateway. Nhưng lưu đệm sự kiện phần mềm và tích hợp PLC thực tế có khác biệt về giao thức, ACK và khả năng thiết bị; không nói đã triển khai gateway công nghiệp chỉ dựa trên kinh nghiệm RabbitMQ.

6.8. Xử lý sự kiện thiết bị đến sai thứ tự thế nào?

Không dùng thứ tự nhận qua mạng làm bằng chứng duy nhất về thứ tự nghiệp vụ.

Dữ liệu nên có: DeviceId, EventId, số thứ tự, mã phiên khởi động, thời gian phát sinh, thời gian nhận và mã lệnh/lần thực hiện. Đồng hồ thiết bị có thể lệch hoặc bị chỉnh lại.

Ví dụ: MES nhận "hoàn tất" số 102 trước "bắt đầu" số 101. Có thể giữ sự kiện chờ trong giới hạn, yêu cầu gửi lại số thiếu hoặc lưu raw rồi xây trạng thái khi đủ điều kiện.

Với snapshot trạng thái, chỉ áp dụng phiên bản mới hơn theo quy tắc. Với sự kiện tăng sản lượng, không bỏ bản cũ chỉ vì đến muộn; nó có thể là một lượng cần tính.

Sequence phải có phạm vi rõ. Sau khởi động lại thiết bị có thể quay về 1, nên chỉ dùng DeviceId + Sequence chưa chắc duy nhất. Dữ liệu không có sequence chỉ cho phép suy luận có giới hạn.

Lưu sự kiện gốc để có thể xử lý lại, giám sát khoảng thiếu và có chính sách cho sự kiện quá muộn thay vì âm thầm bỏ.

---

7. Giao diện Angular / React / TypeScript

7.1. ⭐ Angular khác React như thế nào?

Angular là framework có nhiều cơ chế tích hợp như DI, router, form và HTTP client. React là thư viện xây dựng giao diện theo component; ứng dụng thường chọn thêm router, cách lấy dữ liệu và các thành phần hệ sinh thái.

Khía cạnh	Angular	React
Cấu trúc	Framework với nhiều quy ước và khả năng tích hợp	Linh hoạt, cần thống nhất stack ứng dụng
Giao diện	Template và binding	JSX cùng mô hình component
Phụ thuộc	DI là phần quan trọng	Thường dùng props, context và module
Trạng thái/phản ứng	Có nhiều cơ chế như signals, RxJS theo nhu cầu	State, reducer, context và thư viện bổ sung

Ví dụ MES: Angular phù hợp khi nhóm muốn quy ước thống nhất cho nhiều form nghiệp vụ; React cũng đáp ứng được nếu nhóm thống nhất cách quản lý form, query và phân quyền.

Không nói React luôn nhanh hơn hoặc Angular luôn nặng hơn trong mọi ứng dụng. Hiệu năng phụ thuộc cách render, kích thước dữ liệu và kiến trúc màn hình.

Khi trả lời kinh nghiệm cá nhân, nêu công nghệ đã dùng và một vấn đề đã xử lý; không tự nhận thành thạo cả hai chỉ vì biết so sánh lý thuyết.

Cách mô tả bản thân theo CV: "Thế mạnh chính của tôi là .NET/SQL Server. Tôi có khoảng 2 năm Angular và 1 năm React; vai trò tại TMSS có .NET Core và Angular, còn TYT có Angular/React trong phạm vi công việc."

Booking API và OESMedicalAPI liệt kê React trong stack nhưng vai trò của bạn là backend. Vì vậy không tự nhận toàn bộ giao diện React của hai dự án đó do bạn phát triển.

7.2. Promise khác Observable như thế nào?

Promise biểu diễn một kết quả hoặc lỗi của một hoạt động. Observable biểu diễn một chuỗi thông báo có thể gồm không, một hoặc nhiều giá trị, lỗi và hoàn tất.

Promise executor bắt đầu khi tạo Promise; Observable có thể là cold hoặc hot tùy nguồn, không phải mọi Observable đều chờ subscribe mới bắt đầu tạo dữ liệu.

Ví dụ: lấy chi tiết một lệnh có thể dùng Promise. Theo dõi trạng thái máy hoặc luồng tìm kiếm thay đổi liên tục phù hợp Observable.

```
const results$ = searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term ⇒ api.searchOrders(term))
);
```

switchMap bỏ subscription cũ khi có tìm kiếm mới. Với Angular HttpClient, hủy subscription có thể hủy request phía client; điều đó không bảo đảm server đã dừng một thay đổi nghiệp vụ.

Promise không có cơ chế cancel tích hợp chung; API như fetch có thể dùng AbortController. Observable có unsubscribe, nhưng hiệu quả hủy phụ thuộc nguồn có triển khai teardown hay không.

Bẫy: không dùng switchMap tùy tiện để gửi lệnh trừ tồn, vì việc bỏ subscription không làm giao dịch trên server biến mất.

7.3. Quản lý trạng thái dùng chung trong React hoặc Angular thế nào?

Tách các loại trạng thái trước khi chọn công cụ:

- Trạng thái cục bộ: ô nhập, hộp thoại, lựa chọn tạm.
- Trạng thái điều hướng: bộ lọc cần chia sẻ qua URL.
- Dữ liệu server: danh sách lệnh, chi tiết, tồn và thời gian cập nhật.
- Trạng thái ứng dụng: người dùng, tenant/nhà máy đang chọn.

Ví dụ React: dùng state/reducer cho tương tác cục bộ, context cho dữ liệu dùng chung ít thay đổi, công cụ quản lý truy vấn cho dữ liệu server nếu stack cho phép.

Ví dụ Angular: service cung cấp signals hoặc Observable để chia sẻ trạng thái trong phạm vi; store khi nhiều sự kiện và nhiều màn hình tạo độ phức tạp đủ lớn.

Ví dụ lỗi: người dùng chuyển từ nhà máy A sang B nhưng cache key chỉ có orderId hoặc trạng thái danh sách không được làm mới. Giao diện có thể hiển thị dữ liệu sai phạm vi. Key phải gắn tenant, nhà máy và bộ lọc cần thiết.

Sau mutation, cập nhật hoặc làm mất hiệu lực dữ liệu liên quan. Không sao chép một dữ liệu server sang nhiều store rồi để mỗi nơi tự cập nhật khác nhau.

7.4. ⭐ Xử lý xác thực và phân quyền frontend thế nào?

Frontend hiển thị đúng trạng thái đăng nhập và khả năng thao tác, còn backend là nơi quyết định quyền cuối cùng.

Luồng minh họa: đăng nhập qua hệ thống định danh → thiết lập phiên → tải quyền/phạm vi nhà máy → bảo vệ điều hướng → gọi API → xử lý phiên hết hạn và thiếu quyền.

Angular có route guard; React có thành phần hoặc loader bảo vệ route tùy router. Không dùng quyền từ giao diện làm bằng chứng backend được phép bỏ kiểm tra. Nguồn: giới hạn bảo vệ bằng Angular guard.

Lưu phiên: nếu dùng cookie HttpOnly qua mô hình backend cho frontend, JavaScript không đọc trực tiếp cookie đó; vẫn phải cấu hình SameSite, CSRF và HTTPS theo luồng. Nếu dùng bearer token, chọn nơi lưu và refresh phù hợp, đánh giá rủi ro XSS; không lưu token dài hạn tùy tiện.

Ví dụ: người vận hành thấy nút báo sản lượng nhưng không thấy nút duyệt hủy. Nếu họ tự gọi API hủy, backend vẫn từ chối.

Khi nhiều request cùng gặp hết hạn, phối hợp một lần refresh thay vì tạo hàng loạt refresh cạnh tranh.

Liên hệ thực tế: dùng MasterSSO/RBAC ở TMSS và đăng nhập hợp nhất ở TYT làm ví dụ. Giải thích quyền Host khác quyền đại lý và cách giao diện phản ánh phạm vi đang chọn. Cách lưu token, refresh hoặc cấu hình cookie phải bám luồng thực tế; CV liệt kê SSO không tự xác nhận dùng mô hình backend cho frontend.

7.5. Triển khai HTTP interceptor hoặc lớp gọi API thế nào?

Dùng một nơi thống nhất cho header xác thực, mã truy vết, xử lý lỗi và chính sách timeout phù hợp.

Ví dụ Angular minh họa; AuthSession và apiOrigin là thành phần cấu hình của ứng dụng:

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthSession).accessToken();
  const apiOrigin = inject(ApiConfiguration).origin;
  const target = new URL(req.url, window.location.origin);

  if (!token || target.origin !== apiOrigin) return next(req);

  return next(req.clone({
    setHeaders: { Authorization: "Bearer " + token }
  }));
};
```

Giới hạn đích để không gửi token sang URL bên ngoài. React không có HTTP interceptor mặc định; có thể dùng wrapper fetch hoặc cơ chế của thư viện HTTP đã chọn.

Retry: request đọc có thể retry lỗi tạm thời theo chính sách. Không tự retry mọi POST trừ tồn nếu chưa có idempotency. Với lỗi 403, không refresh token vô hạn vì thiếu quyền không nhất thiết do token hết hạn.

Nếu dùng cookie HttpOnly, không cố đọc token bằng JavaScript; cấu hình credentials và bảo vệ request theo mô hình đó. Nguồn: HTTP interceptor Angular.

7.6. useEffect khác useMemo như thế nào?

useEffect dùng để đồng bộ component với hệ thống bên ngoài, như subscription, kết nối hoặc API theo vòng đời. useMemo lưu kết quả của một phép tính thuần để có thể tái sử dụng khi dependency không đổi.

```
const visibleOrders = useMemo(
  () => orders.filter(x => x.status === selectedStatus),
  [orders, selectedStatus]
);

useEffect(() => {
  const stop = productionFeed.subscribe(factoryId, onProductionChanged);
  return () => stop();
}, [factoryId, onProductionChanged]);
```

Ví dụ trên giả định stop là hàm hủy đăng ký và onProductionChanged có identity được quản lý đúng.

Không gọi API hoặc cập nhật database bên trong useMemo. Giá trị cache của useMemo là tối ưu hiệu năng, không phải cơ chế bảo đảm nghiệp vụ hoặc kho lưu dữ liệu.

Effect cần cleanup cho subscription; dependency sai có thể gây dữ liệu cũ hoặc kết nối lại liên tục. React Strict Mode có thể chạy thêm chu kỳ setup/cleanup trong phát triển để giúp phát hiện lỗi, nên effect phải được viết đúng vòng đời.

Nếu chỉ cần tính giá trị từ props/state, thường tính trực tiếp hoặc memo khi có lý do đo được, không tạo effect chỉ để sao chép state. Nguồn: useEffect, useMemo.

7.7. Xây component tái sử dụng cho ứng dụng doanh nghiệp lớn thế nào?

Tách thành phần trình bày dùng chung khỏi nghiệp vụ màn hình. Component cần API đầu vào/đầu ra rõ, trạng thái loading/empty/error, hỗ trợ bàn phím, nhãn và cách kiểm tra phù hợp.

Ví dụ: bảng dữ liệu dùng chung nhận cột, dòng, trạng thái tải và sự kiện đổi trang. Nó không tự quyết định "lệnh nào được hủy"; quy tắc đó thuộc màn hình hoặc policy nghiệp vụ.

Trong MES, các thành phần hữu ích có thể là ô quét barcode, chọn lô, hiển thị trạng thái máy và nhập lượng theo đơn vị.

Ví dụ ô quét: nhận chuỗi quét và phát sự kiện hoàn chỉnh; màn hình quyết định quét lô vật tư hay serial thành phẩm. Component báo rõ thành công/lỗi, không chỉ đổi màu mà không có chữ.

Tránh một component "đa năng" có hàng chục cờ boolean và logic riêng cho từng module. Chuẩn hóa token giao diện và hành vi, nhưng cho phép ghép nhiều component nhỏ.

Với màn hình xưởng, xem xét thao tác găng tay, kích thước nút, âm thanh/hiển thị phản hồi và chất lượng mạng thực tế.

7.8. Hiển thị trạng thái sản xuất gần thời gian thực thế nào?

Lấy snapshot ban đầu qua API, sau đó nhận thay đổi bằng SignalR/WebSocket hoặc polling theo nhu cầu.

Ví dụ: client đăng ký nhà máy A, nhận sự kiện phiên bản 125 cho dây chuyền L1 và cập nhật dòng tương ứng. Nếu thấy thiếu phiên bản hoặc reconnect, tải snapshot để đồng bộ lại.

Các điểm thiết kế:

- Backend kiểm tra quyền trước khi cho vào nhóm nhận dữ liệu.
- Sự kiện có định danh, phiên bản và thời điểm; client xử lý lặp/cũ.
- Gộp cập nhật khi tần suất cao, tránh render từng điểm dữ liệu máy.
- Hiển thị mất kết nối và thời điểm cập nhật cuối.
- Hủy subscription khi chuyển màn hình.

Ví dụ lỗi: dashboard hiện "đang chạy" từ 30 phút trước nhưng không có dấu hiệu mất kết nối. Người vận hành có thể hiểu nhầm đó là trạng thái hiện tại.

Frontend không quyết định số liệu chính thức bằng cộng dồn message tùy ý. Nếu có bỏ lỡ sự kiện, cần khôi phục từ dữ liệu server có kiểm soát.

---

8. Kiến trúc và thiết kế hệ thống

8.1. ⭐ Giải thích kiến trúc modular monolith bạn đã sử dụng

Câu trả lời đã điều chỉnh để thống nhất CV với mã nguồn:

"Trong TYT, tôi tham gia phát triển hệ thống .NET 9, dùng Aspire và tổ chức theo các domain như Account, Customer và Loyalty. Mỗi domain được chia thành contract, model, repository và dịch vụ, có phân tách command và dữ liệu đọc.

Trong CV tôi dùng cụm modular monolith để mô tả cách tổ chức module. Tuy nhiên, khi nhìn runtime, nhiều Domain Manager thực tế là các ứng dụng riêng, giao tiếp qua gRPC và RabbitMQ. Vì vậy mô tả chính xác hơn cho toàn hệ thống là các dịch vụ theo domain, với code tổ chức theo module/layer."

Nếu người phỏng vấn hỏi modular monolith thực sự là gì: đó thường là một đơn vị triển khai chính, các module cùng process và gọi qua API nội bộ, nhưng vẫn có ranh giới dữ liệu và trách nhiệm rõ.

Ví dụ đối chiếu TYT: AppHost đăng ký AccountManager, CustomerManager và LoyaltyManager riêng. Shared cung cấp interface; client gRPC tạo proxy để gọi implementation trên host tương ứng. Đưa implementation vào cùng process rồi gọi DI mới là một đường gọi nội bộ khác.

Điểm cần sửa cách nói: "hơn 200 project" hoặc "nhiều project kỹ thuật trong solution" không đồng nghĩa "hơn 200 module nghiệp vụ". Một module có thể gồm nhiều project như Commands, Domains, ReadModels, Repository và Manager.

Vai trò cá nhân: CV ghi cùng tham gia thiết kế; nên nói "tôi tham gia thiết kế và phát triển", rồi nêu phạm vi bạn trực tiếp phụ trách. Không tự nhận một mình thiết kế toàn hệ thống.

8.2. CQRS là gì và khi nào nên dùng?

CQRS tách trách nhiệm thay đổi dữ liệu và đọc dữ liệu. Command biểu đạt ý định nghiệp vụ; query trả dữ liệu phục vụ đọc.

Ví dụ:

- Command: xác nhận tiêu hao, hoàn tất công đoạn, duyệt hủy.
- Query: danh sách lệnh, dashboard ca, cây truy xuất.

Có thể dùng chung database nhưng khác model và đường xử lý. Khi cần, mô hình đọc được xây riêng để tổng hợp nhanh. CQRS không bắt buộc RabbitMQ, hai database hoặc event sourcing.

Lợi ích: phía ghi giữ quy tắc nghiệp vụ rõ; phía đọc tối ưu theo màn hình. **Chi phí:** thêm model, mapping, đồng bộ và xử lý dữ liệu đọc có độ trễ nếu tách store.

Ví dụ: dashboard có bảng tổng hợp theo dây chuyền, trong khi command hoàn tất công đoạn vẫn cập nhật dữ liệu gốc theo transaction.

Không áp dụng toàn bộ cơ chế phức tạp cho danh mục CRUD nhỏ nếu không có nhu cầu. Thư mục Commands và ReadModels cho thấy định hướng phân tách, nhưng không tự chứng minh đã có mọi đặc điểm CQRS. Nguồn: mẫu CQRS.

8.3. Monolith khác microservices thế nào?

Monolith có đơn vị triển khai chính hợp nhất. Microservices chia hệ thống thành dịch vụ có trách nhiệm và quyền sở hữu dữ liệu rõ, có khả năng triển khai/vận hành độc lập theo mức độ thiết kế.

Tiêu chí	Monolith có module tốt	Microservices
Giao tiếp nội bộ	Gọi trong process	Qua mạng/message
Giao dịch	Thuận lợi khi cùng database	Cần thiết kế nhất quán liên dịch vụ
Triển khai	Đơn vị lớn hơn	Theo từng dịch vụ
Vận hành	Thường đơn giản hơn	Nhiều dependency và điểm lỗi hơn
Mở rộng	Thường mở rộng cả ứng dụng	Có thể mở rộng từng dịch vụ

Ví dụ: kho chịu tải lớn trong hệ microservices có thể mở rộng riêng. Nhưng nếu mọi request phải đi qua chuỗi năm dịch vụ và các dịch vụ cùng sửa chung bảng, lợi ích có thể bị giảm mạnh.

Nhiều project không quyết định kiến trúc. Nhiều process cũng chưa đủ chứng minh các ranh giới đã tốt. Có thể tồn tại hệ phân tán nhưng phụ thuộc chặt, khiến triển khai vẫn phải đồng loạt.

Lựa chọn dựa trên phạm vi, nhóm phát triển, yêu cầu vận hành và tải thực tế.

8.4. ⭐ Thiết kế hệ nhiều tenant có cô lập dữ liệu thế nào?

Chọn mô hình theo mức cô lập và chi phí: database riêng từng tenant, schema riêng trong những hệ hỗ trợ quản trị phù hợp, hoặc bảng chung có TenantId.

Với bảng chung, TenantId phải tham gia đường đọc, ghi, khóa nghiệp vụ và liên kết dữ liệu. Không chỉ thêm một cột rồi dựa vào developer nhớ lọc.

Ví dụ: unique mã lệnh là (TenantId, OrderCode). Quan hệ từ Operation đến Order phải ngăn tham chiếu tenant khác bằng thiết kế khóa/ràng buộc phù hợp.

Tenant context lấy từ danh tính và phạm vi được backend xác nhận. TenantId do client gửi chỉ là yêu cầu chọn phạm vi, không tự là quyền.

EF Core có global query filter hỗ trợ lọc mặc định, nhưng không bao phủ mọi SQL thô, mọi đường ghi hoặc quyền tắt filter. Dapper/procedure phải truyền và áp dụng tenant tường minh. Nguồn: global query filter.

Phạm vi ngoài SQL: cache key, đường dẫn file, message, background job, xuất báo cáo và nhóm SignalR đều cần tenant.

Kiểm tra: người tenant A dùng Id tài nguyên tenant B ở GET, UPDATE và export phải bị chặn. Không dùng cache/session singleton để giữ "tenant hiện tại" cho mọi request.

Câu trả lời theo CV: "Tôi đã làm nhiều tenant ở OESMedicalAPI và phân tách dữ liệu Host/đại lý trong TMSS. Điều tôi cần bảo vệ là người dùng không đọc hoặc sửa dữ liệu ngoài phạm vi được cấp, kể cả khi họ thay Id trong request."

Với ABP/ASP.NET Zero, nói rõ cơ chế tenant và quyền Host của hệ bạn triển khai. Không mặc định framework tự lọc mọi truy vấn Dapper/procedure hoặc mọi background job. Một nhà máy cũng không nhất thiết tương đương một tenant; có thể là đơn vị bên trong tenant.

8.5. Repository pattern là gì và khi nào hữu ích?

Repository cung cấp giao diện truy cập dữ liệu theo nhu cầu nghiệp vụ, tách caller khỏi chi tiết lưu trữ.

```
public interface IProductionOrderRepository
{
    Task<ProductionOrder?> GetForExecutionAsync(
        Guid tenantId, Guid orderId, CancellationToken ct);
}
```

Ví dụ: handler nhận repository để tải lệnh đúng tenant, thay vì chứa chi tiết tên procedure và mapping SQL. Trong TYT, module thường có interface repository và implementation SQL riêng.

Repository hữu ích khi có truy vấn lặp, mapping phức tạp, ranh giới domain hoặc nhiều chi tiết lưu trữ cần che giấu.

Đánh đổi: EF Core đã cung cấp nhiều khả năng truy cập và theo dõi dữ liệu. Một generic repository chỉ sao chép mọi method của DbSet có thể thêm lớp mà không tạo giá trị.

Repository không tự quyết định transaction cho toàn use case. Nếu tiêu hao cần cập nhật nhiều bảng, cần đơn vị giao dịch bao phủ toàn thao tác. Không để mỗi repository tự commit riêng khiến dữ liệu nghiệp vụ nửa thành công.

Tránh trả IQueryable không kiểm soát rồi để caller tự thay đổi ranh giới tenant hoặc truy vấn tải không giới hạn.

8.6. Thiết kế hệ thống chịu nhiều yêu cầu đồng thời thế nào?

Bắt đầu bằng bất biến và số liệu tải, không bằng việc thêm cache.

Ví dụ tải giả định: 200 trạm báo kết quả cùng lúc, cùng cạnh tranh một số lô vật tư. Phải xác định request/giây, độ trễ chấp nhận và điểm nóng nằm ở đâu.

Các hướng xử lý:

- Câu lệnh cập nhật nguyên tử, unique key và concurrency token.
- Transaction ngắn, thứ tự khóa thống nhất, index theo truy vấn.
- I/O async, giới hạn số công việc đang chạy và connection pool phù hợp.
- API không giữ trạng thái phiên cục bộ nếu cần mở rộng nhiều instance.
- Queue cho bước xử lý nền, có backpressure và theo dõi tuổi công việc.
- Cache dữ liệu đọc phù hợp; không dùng số tồn cache làm quyết định cuối.

Điểm nóng: một dòng tổng tồn có thể trở thành nút tranh chấp dù tăng số API instance. Cần xem mô hình phân bổ theo lô/kho hoặc tuần tự hóa theo khóa nghiệp vụ, nhưng không tùy tiện chia nhỏ làm sai quy tắc.

Kiểm thử nhiều request khác nhau và request lặp, có lỗi giữa chừng. Throughput cao nhưng tồn âm không phải thành công.

8.7. ⭐ Thiết kế MES có lệnh sản xuất, vật tư và traceability

Làm rõ yêu cầu: sản xuất rời rạc hay theo mẻ, theo lô hay serial, số nhà máy, thiết bị, tần suất sự kiện, thời gian offline và hệ thống sở hữu tồn/BOM.

Ranh giới chức năng gợi ý:

1. Dữ liệu kỹ thuật: sản phẩm, BOM, routing, phiên bản.
2. Lệnh và điều phối: phát hành, thực hiện, tạm dừng, hoàn tất.
3. Vật tư: giữ chỗ/cấp phát/tiêu hao/trả lại.
4. Chất lượng: tiêu chí, kiểm tra, cách ly, sửa lại.
5. Truy xuất: quan hệ nguồn–đích, lô, serial và lịch sử.
6. Tích hợp/thiết bị: Inbox, Outbox, gateway và đối soát.
7. Mô hình đọc: tiến độ, WIP và báo cáo.

```
flowchart LR
    ERP[ERP] ⟶ INT[Tích hợp và Inbox]
    INT ⟶ MES[Lệnh và thực thi]
    UI[Giao diện xưởng] ⟶ MES
    DEV[Thiết bị và gateway] ⟶ INT
    MES ⟶ MAT[Vật tư và chất lượng]
    MES ⟶ DB[(Dữ liệu và lịch sử)]
    MAT ⟶ DB
    DB ⟶ OUT[Outbox và worker]
    OUT ⟶ ERP
    OUT ⟶ READ[Mô hình đọc và dashboard]
```

Ví dụ luồng: nhận lệnh 100 sản phẩm, chốt BOM/routing, phân bổ lô, quét vào công đoạn, ghi tiêu hao và liên kết đầu ra, kiểm tra chất lượng, báo hoàn thành và đồng bộ ERP.

Bất biến: không vượt tồn, không tạo trùng lần thực hiện, không bỏ qua bước bắt buộc, không dùng lô bị cấm. Bảo vệ trong database hoặc tại dịch vụ sở hữu, không chỉ trên màn hình.

Có thể bắt đầu bằng modular monolith và worker riêng nếu phù hợp nhóm/tải. Chỉ tách dịch vụ khi có lý do về ownership, triển khai hoặc tải; không cần một microservice cho mỗi bảng.

8.8. Audit và lịch sử sản xuất để tránh mất dữ liệu thế nào?

Audit nghiệp vụ cần được lưu cùng giao dịch thay đổi dữ liệu khi yêu cầu phải đầy đủ. Log console/file bất đồng bộ không phải bằng chứng nguyên tử của nghiệp vụ.

Ví dụ: đổi trạng thái công đoạn cần lưu người thực hiện, thời gian, trạng thái trước/sau, lý do, nguồn, RequestId và phiên bản. Trạng thái mới, audit và Outbox phải cùng thành công hoặc cùng rollback trong đơn vị giao dịch.

Dữ liệu lịch sử ưu tiên bổ sung; sửa sai bằng bản điều chỉnh có liên kết. Không xóa lịch sử lỗi chất lượng chỉ để báo cáo đẹp.

Ngoài thiết kế bảng: phân quyền sửa/xóa, giám sát thay đổi, retention, backup, phục hồi theo thời điểm khi được cấu hình, và diễn tập khôi phục. Dữ liệu chỉ lưu một bản trên một ổ đĩa vẫn có nguy cơ mất.

Nếu có yêu cầu chống sửa mạnh, cân nhắc kho bất biến hoặc cơ chế kiểm chứng phù hợp. Gọi một bảng là append-only trong code chưa ngăn quản trị viên sửa trực tiếp.

Không hứa "không mất dữ liệu tuyệt đối"; xác định rõ phạm vi, điểm ACK/commit, mục tiêu mất dữ liệu chấp nhận và thời gian khôi phục.

Liên hệ CV: OESMedicalAPI có lịch sử cập nhật sản phẩm; TYT có lịch sử giao dịch điểm và luồng đổi voucher. Chọn một ví dụ bạn biết cách lưu thật để giải thích audit. Chưa có bằng chứng thì không nhận đã dùng event sourcing, database bất biến hoặc cơ chế chống sửa đặc biệt.

---

9. Redis / Docker / triển khai / chẩn đoán

9.1. ⭐ Bạn dùng Redis trong dự án như thế nào?

Câu trả lời theo CV:

"Tôi có khoảng 3 năm kinh nghiệm Redis theo CV và đã làm với Redis trong nhiều hệ thống .NET, từ các dự án tại Edsolab đến OESMedicalAPI và các nền tảng Toyota. Một cách sử dụng tôi có thể trình bày là cache dữ liệu đọc để giảm các truy vấn lặp.

Khi dùng cache, tôi quan tâm nguồn dữ liệu chính, cách tạo key theo phạm vi, thời hạn, cách làm mất hiệu lực khi ghi và phương án khi Redis không sẵn sàng. Trong hệ nhiều tenant, cùng một mã dữ liệu không được dùng chung cache key nếu có thể làm lộ hoặc nhầm dữ liệu."

Ví dụ gắn OESMedicalAPI: cache một danh mục hoặc kết quả đọc phù hợp theo tenant; đây là ví dụ áp dụng từ stack CV, chưa xác nhận danh mục cụ thể đã được cache bằng thuật toán nào.

Đối chiếu TYT: RedisCacheService có thao tác cache và namespace theo môi trường/kiểu dữ liệu. Cần xem caller để biết tenant và bộ lọc đã tham gia key chưa.

Không khẳng định đã dùng Redis cho giữ chỗ Booking API, khóa phân tán, phiên đăng nhập hoặc toàn bộ số dư điểm nếu CV chưa nêu cơ chế đó. Booking có giữ chỗ, nhưng công nghệ dùng để hiện thực phải lấy từ code hoặc kinh nghiệm thật.

Liên hệ MES: cache phù hợp cho danh mục và mô hình đọc. Nếu SQL sở hữu tồn, quyết định tiêu hao vẫn phải được bảo vệ tại SQL, không dùng snapshot Redis để trừ vật tư một cách độc lập.

9.2. Bạn biết hoặc đã dùng những chiến lược cache nào?

Chiến lược	Luồng	Điểm cần cân nhắc
Đọc ngoài cache rồi tự nạp	Kiểm tra cache, thiếu thì đọc DB và lưu	Phải xử lý cache cũ và nhiều request cùng nạp
Đọc qua lớp cache	Caller gọi một lớp chịu trách nhiệm nạp	Cần hợp đồng rõ cho lỗi và thời hạn
Ghi qua	Lớp ghi cập nhật nguồn và cache theo cơ chế thiết kế	Hai hệ lưu trữ không tự có transaction chung
Ghi trì hoãn	Đệm ghi rồi chuyển về nguồn sau	Có nguy cơ mất/chậm nếu độ bền không đủ

Ví dụ: cache-aside cho danh mục đơn vị tính. Thiếu cache thì đọc SQL, lưu trong vài phút theo yêu cầu độ tươi; SQL vẫn là nguồn chính.

Để tránh nhiều request cùng nạp khi hết hạn, có thể dùng một tác vụ nạp chung theo key, TTL có độ lệch và giới hạn tải. Trong nhiều instance, cơ chế cục bộ chỉ giảm trùng ở từng instance.

Không cache mọi dữ liệu. Dữ liệu tồn quyết định giao dịch, quyền vừa bị thu hồi hoặc trạng thái an toàn cần chính sách chính xác hơn danh mục. Nguồn: cache-aside Redis.

9.3. Làm mất hiệu lực cache thế nào?

Phải biết dữ liệu nào phụ thuộc vào thay đổi. Với cache-aside, thường commit database rồi xóa key liên quan; lần đọc sau nạp dữ liệu mới.

Ví dụ: sửa tên vật tư M1 thì xóa chi tiết M1 và danh sách chịu ảnh hưởng, theo tenant. Chỉ xóa key chi tiết có thể khiến bảng danh mục vẫn hiện tên cũ.

Khoảng tranh chấp: request A đọc DB cũ; B cập nhật DB và xóa cache; A sau đó ghi dữ liệu cũ vào cache. Vì vậy "xóa sau commit" chưa tự bảo đảm nhất quán mạnh.

Giải pháp tùy nhu cầu: thời hạn ngắn, version trong key/payload, kiểm soát tác vụ nạp, hoặc cơ chế invalidation theo sự kiện. Nếu phát sự kiện xóa cache là bước cần tin cậy, dùng Outbox thay vì gọi rồi bỏ qua lỗi.

Lỗi Redis: dữ liệu cache không thiết yếu có thể đọc từ DB trong giới hạn tải. Nếu mọi request dồn xuống DB khi Redis tắt, cần giới hạn và bảo vệ để tránh sự cố lan rộng.

Không dùng TTL làm câu trả lời duy nhất nếu người dùng cần thấy kết quả thay đổi ngay sau khi thao tác.

9.4. ⭐ Docker image khác container như thế nào?

Image là gói dùng để tạo môi trường chạy, gồm các layer filesystem và thông tin cấu hình. Container là một instance chạy hoặc đã dừng được tạo từ image, với tiến trình và lớp ghi của nó.

Ví dụ: cùng image API MES có thể tạo ba container. Dừng một container không làm hai container còn lại dừng theo.

Image không phải máy ảo chứa toàn bộ hệ điều hành và kernel riêng theo cách hiểu thông thường; container dùng cơ chế cô lập của nền tảng chạy. Docker Desktop có thể sử dụng máy ảo hỗ trợ trên Windows.

Dữ liệu: ghi vào lớp filesystem của container không phải chiến lược lưu bền vững. Khi thay container, dữ liệu cần giữ phải ở volume hoặc hệ lưu trữ ngoài được thiết kế.

Triển khai: build tạo image; registry lưu image; môi trường chạy tạo container từ image. Tag có thể bị thay đổi nội dung, nên bản phát hành cần tham chiếu phiên bản hoặc digest kiểm soát được.

Không cài sửa thủ công trong container production rồi coi đó là bản sửa lâu dài; lần tạo lại từ image cũ sẽ mất thay đổi.

Theo CV: bạn ghi khoảng 2 năm Docker. Có thể nói về khái niệm, đóng gói và cách đã sử dụng thật. Chưa có thông tin về Kubernetes hay trách nhiệm quản trị cluster thì không tự thêm vào kinh nghiệm.

9.5. Cấu hình ứng dụng .NET bằng Docker Compose thế nào?

Compose mô tả nhiều dịch vụ, mạng, biến môi trường, healthcheck và volume. Ví dụ dưới đây là cấu hình minh họa độc lập, **không dùng lại config key của TYT**:

```
services:
  api:
    image: "${MES_API_IMAGE:?Cần đặt MES_API_IMAGE}"
    ports:
      - "8080:8080"
    environment:
      ASPNETCORE_HTTP_PORTS: "8080"
      Cache__Redis: "redis:6379"
    depends_on:
      redis:
        condition: service_healthy

  redis:
    image: "${REDIS_IMAGE:?Cần đặt REDIS_IMAGE}"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 10
```

Đặt hai biến image bằng phiên bản/digest đã chọn. Cache__Redis chỉ có tác dụng nếu ứng dụng đọc key đó. Database có thể là dịch vụ riêng hoặc hệ thống ngoài; ví dụ không giả định tự tạo SQL Server.

Điểm mạng: trong container API, localhost trỏ tới chính container API. Redis cùng mạng Compose được gọi bằng tên redis.

Khởi động: depends_on theo service_healthy hỗ trợ chờ healthcheck ban đầu; ứng dụng vẫn cần xử lý dependency mất kết nối về sau. Cấu hình secret lấy từ cơ chế quản lý môi trường phù hợp. Nguồn: thứ tự khởi động Compose.

Phân biệt hai công cụ trong hồ sơ: .NET Aspire của TYT điều phối các ứng dụng được khai báo trong AppHost; Docker Compose là một cách mô tả môi trường container. Có kinh nghiệm Aspire không tự chứng minh bạn đã container hóa hoặc triển khai production toàn bộ TYT bằng Compose.

9.6. CI/CD pipeline là gì?

CI tích hợp thay đổi thường xuyên và kiểm tra tự động. CD là chuỗi chuẩn bị/triển khai bản phát hành; có tổ chức giữ bước duyệt trước production, có tổ chức tự động triển khai theo chính sách.

Luồng ví dụ:

```
Thay đổi code
 → khôi phục dependency và build
 → kiểm tra cần thiết
 → tạo artifact/image
 → triển khai môi trường kiểm tra
 → xác nhận
 → triển khai production theo quy trình
 → kiểm tra hoạt động và theo dõi
```

Artifact nên được tạo một lần rồi chuyển qua môi trường, thay vì build lại code khác nhau ở mỗi nơi.

Ví dụ MES: thay đổi API phải tương thích với các màn hình và gateway chưa cập nhật cùng lúc. Thay đổi database nên theo hướng mở rộng trước, chuyển ứng dụng rồi dọn phần cũ khi an toàn.

Rollback: trả image cũ không luôn hoàn tác schema hoặc dữ liệu. Cần hiểu tương thích và có kế hoạch phục hồi riêng.

Chỉ nói "tôi xây pipeline" nếu thực sự làm. Nếu bạn chủ yếu dùng pipeline có sẵn, hãy nêu cách đọc lỗi, xác nhận artifact và phối hợp triển khai.

Căn cứ CV: tại Edsolab và Twenty-Tech có triển khai, theo dõi và hỗ trợ khách hàng. CV không mô tả bạn đã thiết kế pipeline nào. Nếu dùng pipeline có sẵn, trả lời theo vai trò đó; nếu có tự xây, bổ sung hệ CI/CD và các bước bạn trực tiếp cấu hình.

9.7. ⭐ Điều tra API production đột ngột chậm thế nào?

Đầu tiên xác định ảnh hưởng: endpoint nào, nhà máy nào, từ khi nào, bao nhiêu request và p95/p99 thay đổi ra sao.

Quy trình điều tra:

1. Xem thay đổi triển khai/cấu hình và tải tại thời điểm bắt đầu.
2. Theo TraceId tách thời gian tại API, SQL, Redis và dịch vụ ngoài.
3. Kiểm tra CPU, RAM, GC, ThreadPool, connection pool và số request đang xử lý.
4. Xem SQL waits, blocking, deadlock và execution plan có thay đổi.
5. Kiểm tra backlog và tuổi message nếu API phụ thuộc dữ liệu nền.
6. Chọn biện pháp giảm ảnh hưởng có căn cứ rồi xác nhận bằng số liệu.

Ví dụ: API trước mất 300 ms, nay 8 giây. Trace cho thấy 7,5 giây chờ SQL lock do một báo cáo giữ transaction lâu. Thêm API instance không giải quyết nguyên nhân.

Nếu phải khôi phục nhanh, cân nhắc rollback bản mới hoặc tạm giảm workload không thiết yếu theo quy trình vận hành. Thu thập bằng chứng trước restart khi có thể, vì restart có thể làm mất trạng thái điều tra.

Không kết luận lỗi mạng, thiếu RAM hoặc thiếu index chỉ từ một request timeout.

Ví dụ phù hợp hồ sơ: chọn API danh sách/truy vấn sản phẩm, lịch sử điểm hoặc nghiệp vụ TMSS mà bạn từng hỗ trợ. Twenty-Tech và Edsolab có phần monitoring/hỗ trợ trong CV, nhưng không có sự cố và thời gian xử lý cụ thể; giữ ví dụ số liệu phía trên là giả định.

9.8. Phát hiện rò rỉ bộ nhớ hoặc CPU cao thế nào?

Tách hai hướng:

Bộ nhớ: theo dõi managed heap, tốc độ cấp phát, GC, bộ nhớ tiến trình và giới hạn container. RAM cao chưa chắc là leak; runtime có thể giữ vùng nhớ để tái sử dụng hoặc có native allocation.

CPU: dùng counters xác nhận tải CPU, sau đó lấy trace trong khoảng đại diện để tìm stack nóng. CPU thấp nhưng API chậm có thể là chờ I/O, khóa hoặc thiếu thread.

Ví dụ lệnh chẩn đoán, chỉ chạy theo môi trường và quyền vận hành phù hợp:

```
dotnet-counters monitor --process-id <PID>
dotnet-trace collect --process-id <PID>
dotnet-dump collect --process-id <PID>
```

Ví dụ nguyên nhân: singleton giữ Dictionary tăng không giới hạn theo serial; event handler không hủy đăng ký; queue trong RAM không giới hạn; vòng retry lỗi không có chờ.

Với memory leak, so các mẫu heap và xem đường giữ object, không chỉ object nào lớn nhất. Dump có thể chứa dữ liệu nhạy cảm và gây chi phí tài nguyên, nên cần kiểm soát nơi thu thập/lưu.

Sau sửa, đo lại dưới tải tương tự; một lần restart làm RAM giảm chưa chứng minh leak đã được sửa. Nguồn: công cụ chẩn đoán .NET, điều tra memory leak.

Liên hệ Twenty-Tech: công việc crawler/làm sạch dữ liệu bằng C#/Python là một ví dụ để nói về bộ đệm không giới hạn, dữ liệu tăng dần và vòng lặp xử lý. Đây là các rủi ro để phân tích, chưa phải bằng chứng bạn từng gặp leak hoặc CPU cao do đúng nguyên nhân này.

---

10. Hành vi và xử lý tình huống

10.1. ⭐ Kể một sự cố production bạn đã xử lý

Căn cứ CV: bạn có kinh nghiệm triển khai, theo dõi và hỗ trợ khách hàng tại Edsolab/Twenty-Tech, đồng thời làm các thành phần đồng bộ/worker ở TMSS/TYT. CV không kể một sự cố production cụ thể, vì vậy chưa thể viết thành một câu chuyện đã xảy ra.

Khung trả lời cá nhân hóa, cần điền bằng sự cố thật:

"Trong dự án [TMSS, TYT hoặc dự án thực tế khác], tôi trực tiếp hỗ trợ khi [triệu chứng thật]. Trách nhiệm của tôi là [phần bạn thực hiện].

Tôi bắt đầu từ [log, mã giao dịch hoặc dữ liệu thực tế], phân biệt lỗi ở giao diện, API, database hay luồng đồng bộ. Tôi xác định [nguyên nhân có bằng chứng] rồi phối hợp [biện pháp khôi phục được phép].

Sau khi khôi phục, tôi kiểm tra [kết quả cụ thể], xử lý các bản ghi cần đối soát và bổ sung [cải tiến thật nếu có]."

Chủ đề phù hợp để bạn nhớ lại sự cố:

- TMSS/TYT: worker không chạy, thông điệp chờ, trạng thái chưa đồng bộ, lỗi quyền đại lý hoặc truy vấn chậm.
- Twenty-Tech: tác vụ thu thập dữ liệu lỗi, dữ liệu đầu vào thay đổi hoặc tiến trình cần hỗ trợ vận hành.
- Edsolab: lỗi chuyển báo cáo, lịch/thông báo hoặc vấn đề sau triển khai.

Các ý trên là câu hỏi gợi nhớ, không phải các sự cố được CV xác nhận.

Ví dụ luyện tập giả định: hệ nguồn đã commit nhưng hệ nhận chưa cập nhật. Cần kiểm tra Outbox/Inbox và dữ liệu gốc trước khi cho người dùng làm lại. Chỉ kể theo thì quá khứ nếu đó thực sự là việc bạn đã xử lý.

Không tự thêm thời gian khôi phục, phần trăm cải thiện hay khẳng định không mất dữ liệu. Một câu chuyện nhỏ có vai trò và bằng chứng rõ sẽ thuyết phục hơn việc gán mọi công nghệ trong CV vào một sự cố hư cấu.

10.2. Bạn quản lý công việc khi yêu cầu thay đổi thế nào?

Cách trả lời gắn với phạm vi CV:

"Các dự án tôi làm có nhiều quy tắc nghiệp vụ, như báo giá/quyết toán ở TMSS và điểm/voucher ở TYT. Khi yêu cầu thay đổi, tôi cần xác nhận phần thay đổi, lý do và tác động đến dữ liệu, giao diện, quyền và tích hợp trước khi điều chỉnh công việc.

Tôi trao đổi với người phụ trách yêu cầu để thống nhất ưu tiên và tiêu chí nghiệm thu; phần đang làm hoặc đã triển khai cần được đánh giá tương thích. Khi làm việc với QA hoặc khách hàng, tôi dùng ví dụ dữ liệu cụ thể để xác nhận cách xử lý."

Ví dụ minh họa theo TYT: đổi chính sách hết hạn điểm. Không chỉ sửa một ngày trên màn hình; cần biết áp dụng cho điểm mới hay cả điểm đã có, ảnh hưởng voucher đã đổi và sự kiện đang chờ đồng bộ thế nào. CV không xác nhận đã có đúng lần thay đổi này.

Liên hệ mục tiêu dài hạn: cách làm có đánh giá tác động giúp bạn phát triển từ xử lý một màn hình sang hiểu giải pháp toàn luồng.

Nếu được hỏi "kể lần thực tế", chọn thay đổi bạn đã làm, nêu phiên bản yêu cầu ban đầu, điều gì thay đổi và kết quả. Không tự nhận đã quản lý backlog hoặc quyết định lịch của cả nhóm nếu vai trò thực tế không có.

10.3. Bạn từng bất đồng với giải pháp kỹ thuật của nhóm chưa?

CV chưa ghi một lần bất đồng cụ thể. Vì vậy phần dưới là cách tiếp cận và ví dụ luyện tập, không phải thành tích hay sự kiện quá khứ.

Câu trả lời về phương pháp:

"Khi không đồng ý với một giải pháp, tôi cố gắng hiểu mục tiêu của đồng đội trước. Sau đó tôi trình bày rủi ro bằng một tình huống cụ thể, so sánh phương án theo độ đúng, hiệu năng, độ phức tạp và khả năng vận hành. Nếu cần, tôi đề nghị một thử nghiệm nhỏ hoặc đối chiếu đoạn code. Khi nhóm thống nhất, tôi thực hiện theo quyết định và ghi lại đánh đổi."

Ví dụ gần kinh nghiệm trong CV: chống đổi voucher trùng bằng khóa C# có thể không đủ khi dịch vụ chạy nhiều instance. Bạn có thể trình bày vì sao cần bảo vệ tại database và kiểm tra bằng hai request đồng thời.

Một ví dụ khác: việc có interface Shared không có nghĩa gọi được implementation trên server khác mà không qua mạng. Có thể dùng cách phân tích này để trao đổi kỹ thuật, nhưng không coi cuộc trao đổi hiện tại là một lần bất đồng trong nhóm cũ.

Khi điền câu chuyện thật: nêu đề xuất ban đầu, điểm bạn lo ngại, bằng chứng, ý kiến của đồng đội và quyết định cuối. Thừa nhận phần bạn đã thay đổi quan điểm nếu có.

10.4. Xử lý lỗi nghiêm trọng do khách hàng báo thế nào?

Câu trả lời theo nền tảng hỗ trợ khách hàng trong CV:

"Tôi có kinh nghiệm triển khai, theo dõi và hỗ trợ khách hàng ở các dự án tại Edsolab và Twenty-Tech. Khi nhận lỗi nghiêm trọng, tôi muốn xác định ảnh hưởng và thông tin đủ để điều tra, thay vì sửa theo mô tả chung chung.

Tôi hỏi thời điểm, mã nghiệp vụ, môi trường, tài khoản hoặc phạm vi đơn vị và các bước dẫn tới lỗi. Với hệ Toyota, tôi còn cần phân biệt lỗi riêng một đại lý, một luồng tích hợp hay toàn hệ thống. Sau đó tôi phối hợp khôi phục theo quy trình và xác nhận bằng dữ liệu thật."

Ví dụ minh họa: khách báo một giao dịch Loyalty chưa phản ánh sang hệ thống liên quan. Cần xem nghiệp vụ đã commit chưa, Outbox có bản ghi không, worker đã gửi chưa, Inbox nhận thế nào và dữ liệu đọc có đang cũ. Đây là hướng điều tra, không phải sự cố CV đã ghi.

Không đề nghị khách bấm đổi hoặc gửi lại hàng loạt trước khi biết cơ chế chống trùng. Nếu cần sửa dữ liệu, phải xác định trạng thái gốc, được phép thực hiện và có audit.

Điểm nói rõ: kinh nghiệm hỗ trợ được CV xác nhận; nguyên nhân và kết quả của một sự cố cụ thể cần bạn bổ sung. Không tự nhận đã trực vận hành mọi thành phần production của TMSS/TYT.

10.5. ⭐ MES ngừng hoạt động trong lúc nhà máy đang sản xuất thì làm gì?

Ưu tiên phối hợp người phụ trách vận hành nhà máy để đánh giá mức ảnh hưởng và áp dụng quy trình đã được duyệt.

Trình tự:

1. Xác định phạm vi: toàn MES, một trạm, kết nối thiết bị hay chỉ dashboard.
2. Xác nhận tình trạng dữ liệu tại PLC/gateway và khả năng lưu đệm.
3. Áp dụng chế độ dự phòng được phê duyệt: ghi tại edge hoặc biểu mẫu có định danh, nếu quy trình cho phép.

4. Điều tra dependency và khôi phục dịch vụ theo kế hoạch.
5. Nạp lại dữ liệu có chống trùng, xử lý thứ tự và đối soát trước khi chốt ca.

Ví dụ: API trung tâm tạm ngừng nhưng gateway vẫn lưu kết quả. Nhà máy có thể tiếp tục theo phạm vi được phê duyệt. Nếu MES bắt buộc xác nhận vật tư/chất lượng trước bước tiếp theo mà không có cơ chế dự phòng, không tự chỉ đạo bỏ qua kiểm soát.

Khi phục hồi, không chỉ thấy màn hình lên là kết thúc. Cần kiểm tra số bản ghi đệm, khoảng sequence thiếu, kết quả tích hợp và sai lệch sản lượng/tồn.

10.6. Bạn có sẵn sàng đi công tác tại nhà máy khách hàng không?

Đây là câu hỏi cam kết cá nhân; cần trả lời theo khả năng thật.

Nếu sẵn sàng:

"Tôi sẵn sàng đi công tác khi công việc cần khảo sát, triển khai hoặc hỗ trợ tại nhà máy. Tôi muốn thống nhất trước tần suất, địa điểm, thời gian báo trước và phạm vi hỗ trợ để sắp xếp phù hợp."

Nếu có giới hạn:

"Tôi có thể đi công tác trong [phạm vi thực tế]. Với đợt dài hơn hoặc lịch phát sinh gấp, tôi cần được trao đổi trước để sắp xếp."

Ví dụ giá trị tại hiện trường: trực tiếp quan sát cách công nhân quét mã và xử lý hàng lỗi có thể phát hiện vấn đề không thấy trên tài liệu, như vị trí đặt máy quét hoặc quy trình đổi ca.

Không nên hứa trực 24/7 hoặc đi bất kỳ lúc nào nếu không thể đáp ứng. Sự rõ ràng giúp hai bên đánh giá phù hợp hơn một lời đồng ý không có điều kiện thực tế.

10.7. Giải thích vấn đề kỹ thuật cho người vận hành không chuyên thế nào?

Nói triệu chứng, ảnh hưởng và thao tác cần làm bằng ngôn ngữ hiện trường. Không bắt đầu bằng tên exception, thuật ngữ broker hoặc chi tiết framework.

Ví dụ: thay vì "consumer chưa ACK vì transaction timeout", nói:

"Lần quét của anh/chị đã được lưu, nhưng hệ thống đang chậm ở bước cập nhật kết quả. Anh/chị giữ mã giao dịch này và chưa cần quét lại. Tôi đang kiểm tra; màn hình sẽ hiển thị khi xử lý xong."

Chỉ nói "đã được lưu" nếu đã có bằng chứng. Nếu chưa xác định, nói: "Tôi đang kiểm tra lần quét có được lưu chưa; trước khi quét lại, tôi sẽ xác nhận để tránh trùng."

Dùng một ví dụ cụ thể tại trạm, chỉ dẫn từng bước và yêu cầu người dùng xác nhận lại phần quan trọng. Trạng thái trên màn hình cần dễ hiểu: đang gửi, đã nhận, đã xử lý, cần hỗ trợ.

Khi cập nhật sự cố, nêu phạm vi ảnh hưởng, phương án hiện tại và mốc cập nhật tiếp theo; không hứa thời gian sửa chắc chắn khi chưa biết nguyên nhân.

10.8. Học nghiệp vụ mới chưa từng làm như thế nào?

Câu trả lời theo quá trình dự án trong CV:

"Tôi đã làm qua những lĩnh vực khá khác nhau: các hệ thống báo cáo, lịch họp và phản ánh kiến nghị tại Edsolab; thu thập/làm sạch dữ liệu tại Twenty-Tech; sau đó là TMSS/TYT cho Toyota. Vì vậy, khi vào một lĩnh vực mới, tôi không chỉ học tên màn hình mà cố gắng hiểu dữ liệu và quy trình từ đầu đến cuối.

Tôi chọn một luồng điển hình, trao đổi với người làm nghiệp vụ để xác định ai thao tác, trạng thái thay đổi ra sao, dữ liệu nào là nguồn chính và khi nào phải từ chối. Tôi ghi lại thuật ngữ và dùng ví dụ để kiểm tra hiểu biết trước khi đưa vào API hoặc database.

Với MES, tôi sẽ theo một lệnh từ ERP xuống xưởng, qua BOM/routing, cấp vật tư, thực hiện, kiểm tra và báo kết quả. Phần giao dịch và tích hợp là nơi tôi có thể sử dụng kinh nghiệm hiện có; nghiệp vụ sản xuất và thiết bị là phần cần học cùng BA

và người vận hành.”

Ví dụ: từ quy trình chuyển/giao báo cáo ở Edsolab, bạn có kinh nghiệm tiếp cận các trạng thái và quyền thao tác. MES cũng có trạng thái, nhưng điều kiện chuyển còn liên quan vật tư, chất lượng và thực tế tại trạm; không sao chép nguyên quy tắc cũ.

Khi cần kể cách học thật, chọn một quy tắc của TMSS hoặc TYT mà bạn từng phải làm rõ, rồi nói bạn đã hỏi ai và điều chỉnh implementation thế nào. Không tự thêm một chuyến khảo sát nhà máy chưa từng thực hiện.

Phụ lục A. Một tình huống xuyên suốt để luyện hỏi sâu

A.1. Bối cảnh giả định

Nhà máy có lệnh SX-01 sản xuất 100 bộ linh kiện. Một bộ dùng 2 chi tiết A và 4 vít B. Quy trình gồm gia công, lắp ráp và kiểm tra cuối. Vật tư theo lô, thành phẩm theo serial.

Đây là ví dụ thiết kế cho phỏng vấn, không mô tả dữ liệu hoặc hệ thống production của người dùng.

A.2. Luồng bình thường

1. ERP gửi lệnh với EventId E-001, BOM phiên bản 3 và routing phiên bản 2.
2. MES lưu Inbox với khóa chống trùng, kiểm tra danh mục và chấp nhận lệnh.
3. Hệ thống sở hữu tồn giữ chỗ vật tư theo lệnh, sau đó cấp vật tư ra xưởng.
4. Trạm lắp ráp nhận serial S001 và quét lô A01/B01.
5. API kiểm tra tenant, trạm, trạng thái công đoạn, lô và lượng.
6. Một đơn vị giao dịch ghi kết quả, tiêu hao theo mô hình tồn đã chọn, lịch sử và Outbox.
7. Worker gửi kết quả sang hệ thống liên quan, giữ EventId ổn định qua retry.
8. Dashboard nhận cập nhật; ERP phản hồi đã áp dụng nghiệp vụ theo hợp đồng.

A.3. Các mã định danh không nên dùng lẫn

Định danh	Mục đích
TenantId / FactoryId	Phạm vi dữ liệu và quyền
ProductionOrderId / mã SX-01	Lệnh sản xuất
OperationId / ExecutionId	Công đoạn và lần thực hiện cụ thể
RequestId	Một yêu cầu nghiệp vụ và các lần gửi lại
EventId	Một sự kiện logic qua các lần giao
DeviceId + phiên khởi động + sequence	Nhận diện sự kiện thiết bị theo quy ước
TraceId	Theo dõi đường đi kỹ thuật, không thay khóa idempotency
RowVersion / AggregateVersion	Phát hiện xung đột hoặc thứ tự phiên bản

Một lần retry có thể mang TraceId khác nhưng vẫn phải giữ RequestId/EventId cần thiết. Không dùng một mã trace tạm thời làm khóa duy nhất cho mọi nghiệp vụ.

A.4. Ma trận lỗi và cách giải thích

Điểm lỗi	Kết quả có thể xảy ra	Cách bảo vệ
Client gửi lại vì mất response	Nghiệp vụ đầu có thể đã commit	RequestId, fingerprint và kết quả đã lưu

Điểm lỗi	Kết quả có thể xảy ra	Cách bảo vệ
Hai trạm tiêu cùng phần tồn	Cùng nhìn thấy đủ trước khi ghi	UPDATE có điều kiện hoặc phân bổ nguyên tử
SQL lỗi trước commit	Dữ liệu chưa hoàn tất	Rollback toàn bộ đơn vị giao dịch
Commit xong, broker ngừng	Hệ nhận chưa biết thay đổi	Outbox còn pending và retry
Broker nhận, worker chết trước đánh dấu	Message có thể gửi lại	EventId ổn định và consumer idempotent
Inbox đã lưu nhưng worker chưa xử lý	Broker có thể đã ACK	Worker Inbox, retry và giám sát tuổi bản ghi
Nghiệp vụ nhận đã xong, mark processed lỗi	Có thể xử lý lại	Transaction chung hoặc idempotency nghiệp vụ
Worker quá hạn lease	Hai worker có thể thực thi chồng	Heartbeat/token và fencing/idempotency ở nơi ghi
Gateway mất kết nối MES	Chậm đồng bộ	Bộ đệm bền vững, replay và đối soát
Trình duyệt mất sự kiện dashboard	Hiển thị cũ	Snapshot, cursor/phiên bản và dấu mất kết nối

A.5. Phân biệt ba loại "trùng"

- Trùng truyền nhận: cùng EventId được giao nhiều lần.
- Trùng yêu cầu: người dùng gửi lại cùng RequestId.
- Trùng nghiệp vụ: hai RequestId khác nhau đều muốn hoàn tất cùng serial/công đoạn.

Một unique key trên EventId không tự chặn loại thứ ba. Phải có thêm ràng buộc nghiệp vụ đúng phạm vi, ví dụ serial + công đoạn + lần thực hiện được phép.

A.6. Khi nào dùng lạc quan, cập nhật nguyên tử hoặc tuần tự hóa?

- Chỉnh thông tin lệnh ít xung đột: concurrency token để báo dữ liệu đã thay đổi.
- Trừ một dòng tồn: UPDATE số lượng có điều kiện trong transaction.
- Phân bổ nhiều lô: transaction, thứ tự khóa và index phù hợp; xử lý retry.
- Tài nguyên cực nóng: cân nhắc hàng đợi theo khóa nghiệp vụ hoặc thiết kế phân bổ khác, nhưng phải giữ semantics và khả năng khôi phục.

Không có một loại khóa giải quyết mọi bài toán. Khóa trong RAM, khóa Redis và khóa SQL có phạm vi và điểm lỗi khác nhau.

Phụ lục B. Căn cứ source TYT và phạm vi kết luận

Các liên kết dưới đây là đường dẫn tương đối từ tài liệu tại thư mục gốc repo:

Thành phần đã đọc	Điều có thể xác nhận từ source	Điều chưa tự được chứng minh
AppHost	Đăng ký nhiều ứng dụng theo domain và endpoint riêng	Topology và chính sách deploy production
Contract Loyalty	Interface gRPC code-first	Interface tự thực thi nghiệp vụ
Đăng ký client	Tạo client qua AddCodeFirstGrpcClient	Mọi caller đều gọi từ xa trong mọi trường hợp
Host Loyalty	DI implementation và map endpoint gRPC	Toàn hệ thống đã độc lập về dữ liệu
Transactional store	Một số phương thức ghi aggregate/lịch sử và Outbox qua transaction	Mọi bước của mọi use case đều nằm trong cùng transaction
Outbox publisher	Worker claim, gửi và cập nhật kết quả	Đã kiểm chứng broker confirm và tất cả điểm gián đoạn

Thành phần đã đọc	Điều có thể xác nhận từ source	Điều chưa tự được chứng minh
Outbox claim	Claim nguyên tử và thời hạn khóa	Độc quyền thực thi sau khi lease hết hạn
Consumer ngoài	Lưu payload và định danh vào Inbox	Mọi tác dụng phụ cuối đã cùng transaction với dấu hoàn tất
Inbox processor	Dispatch, retry và trạng thái lỗi	Nghiệp vụ chỉ xảy ra một lần trên mọi đường chạy
Schema Inbox	Script có unique index MessageKey	Script đã được áp dụng đúng ở database production
Cache Redis	Thao tác cache và namespace key	Mọi cache key có cô lập tenant và invalidation đúng

Những câu trả lời mô tả thiết kế mới như REST theo tài nguyên, ProblemDetails hoặc cấu trúc bảng MES là phương án minh họa. Chúng không khẳng định repo hiện tại đã triển khai những phương án đó.

Phụ lục C. Những câu dễ nói sai khi phỏng vấn

Phát biểu dễ sai	Cách nói chính xác hơn
Có nhiều project nên là microservices	Phải xét process, ownership dữ liệu và khả năng triển khai
Shared có interface nên không cần gọi mạng	Interface là hợp đồng; implementation ở process khác vẫn cần transport
async tạo thread mới	async/await tổ chức việc chờ; không tự tạo thread
Singleton tự thread-safe	Phải thiết kế và kiểm chứng an toàn đồng thời
Transaction tự ngăn mọi race condition	Cần câu lệnh, khóa, phiên bản và ràng buộc phù hợp
ROWLOCK chắc chắn không khóa bảng	Không bảo đảm loại bỏ escalation
READPAST bỏ qua mọi khóa	Có giới hạn, không bỏ qua mọi page lock
NOLOCK là cách đọc nhanh mà vẫn đúng	Có thể đọc dữ liệu không nhất quán
Có Outbox là không gửi trùng	Vẫn có khoảng gửi thành công nhưng chưa đánh dấu
Có Inbox là nghiệp vụ luôn đúng một lần	Cần ranh giới giao dịch hoặc idempotency cho hiệu ứng
Publisher confirm nghĩa consumer đã làm xong	Confirm chỉ xác nhận phần trách nhiệm của broker
MQTT QoS 2 làm SQL chỉ cập nhật một lần	Phạm vi giao thức không tự bao phủ giao dịch SQL
Quét barcode là đã có traceability	Cần lưu quan hệ đầu vào–đầu ra và ngữ cảnh
Có đủ tồn thì luôn được tiêu hao	Còn điều kiện chất lượng, giữ chỗ, định mức và quyền
Container chạy nghĩa ứng dụng sẵn sàng	Cần readiness/health và dependency thích hợp
Restart hết chậm nghĩa đã sửa nguyên nhân	Cần bằng chứng và theo dõi sau khôi phục

Phụ lục D. Cách luyện trả lời

1. Luyện các câu ⭐ trước, nhưng đọc phần giải thích để trả lời được câu hỏi tiếp theo.
2. Chuẩn bị ba câu chuyện thật: một chức năng bạn tự chủ, một sự cố và một lần phối hợp thay đổi yêu cầu.
3. Với mỗi câu chuyện, ghi vai trò của bạn, quyết định, đánh đổi và bằng chứng. Không dùng số liệu minh họa trong tài liệu như thành tích.
4. Với câu thiết kế, bắt đầu bằng giả định, chỉ ra nơi sở hữu dữ liệu và bất biến phải bảo vệ.

5. Luyện vẽ luồng một request, một transaction và một message; nói rõ chuyện gì xảy ra nếu lỗi ở từng đoạn.
6. Nếu chưa có kinh nghiệm thiết bị hoặc MES, nói phần chưa làm và trình bày phương pháp tiếp cận cụ thể.

Phụ lục E. Bản đồ kể kinh nghiệm cá nhân theo CV

E.1. Chọn đúng dự án cho từng loại câu hỏi

Chủ đề phỏng vấn	Dự án nên ưu tiên	Điều nên chuẩn bị để hỏi sâu
Giới thiệu và dự án gần nhất	TMSS + TYT tại Nexus	Phạm vi hai hệ thống, vai trò và nhóm 8/12 người
.NET/Angular và nghiệp vụ đại lý	TMSS	Một luồng khách hàng/xe, báo giá, quyết toán hoặc phụ tùng
Đa tenant	TMSS, OESMedicalAPI	Tenant context, quyền Host, truy vấn đọc/ghi và dữ liệu nền
RabbitMQ, Inbox/Outbox, worker	TMSS; dùng source TYT để đối chiếu khi phù hợp	Điểm commit, điểm ACK, khóa nhận việc và chống xử lý lặp
Giao dịch điểm, voucher, đồng thời	TYT	Trạng thái, số dư/hạn mức, hết hạn, hoàn/hủy và đồng bộ
gRPC và Shared	TYT	Interface, client proxy, endpoint host và lỗi mạng
Giữ chỗ và hết hạn	Booking API	Thời điểm giữ, xác nhận/hủy, tranh chấp và tiến trình giải phóng
Danh mục động và lịch sử	OESMedicalAPI	Kiểu dữ liệu, truy vấn, phân trang, quyền và audit
Quy trình/chuyển trạng thái	Báo cáo điện tử, Phản ánh kiến nghị	Ai được giao/chuyển, trạng thái hợp lệ và lịch sử
Lịch và tác vụ nhắc	Họp thông minh; worker TMSS	Lịch lặp, thời điểm chạy, gửi lặp, lỗi và retry
Dữ liệu đầu vào và vận hành	Twenty-Tech	Quy trình thu thập/làm sạch, triển khai và hỗ trợ thật
Docker và CI/CD	Docker trong danh sách kỹ năng; hỗ trợ triển khai tại các công ty	Phân biệt đã dùng công cụ với tự thiết kế hạ tầng/pipeline
MES/PLC	Phương án tiếp cận và kiến thức trong tài liệu	Chỉ bổ sung trải nghiệm trực tiếp nếu thực tế có ngoài CV

E.2. Chín ý quan trọng khi người phỏng vấn bám từng dòng CV

1. **Khoảng 5 năm:** dùng đúng con số CV và không cộng thời gian freelancer song song.
2. **Tham gia thiết kế:** giải thích phần bạn trực tiếp quyết định, phần làm cùng nhóm và phần kế thừa.
3. **Inbox/Outbox:** vẽ đúng một transaction, một worker và một consumer; nói rõ điểm vẫn có thể xử lý lại.
4. **Chạy nhiều instance:** phân biệt khóa trong process với claim/khóa tại database; nêu giới hạn lease.
5. **Dữ liệu lớn:** chọn truy vấn và cách đo thật; CV không có số dòng hay mức cải thiện để tự suy ra.
6. **SSO nhiều bên:** trình bày một luồng bạn nắm rõ, phân biệt nguồn danh tính, token ứng dụng và quyền nghiệp vụ. Không gộp mọi provider thành cùng một giao thức.
7. **Kiến trúc TYT:** chia module ở code không tự làm toàn runtime thành một ứng dụng duy nhất; 200+ project không phải 200+ nghiệp vụ.
8. **React trong stack:** vai trò backend ở Booking/OES không chứng minh bạn sở hữu frontend; kinh nghiệm React nên gắn đúng phần đã làm.
9. **MES/ERP/PLC:** nói rõ kỹ năng chuyển giao từ TMSS/TYT và phần nghiệp vụ mới cần học; không gọi DMS là MES.

E.3. Thông tin vẫn cần bạn xác nhận

- Thời điểm và lý do thực tế kết thúc công việc tại Nexus; cách hiểu chữ “hiện tại” ở hai dự án.
- Vì sao TMSS có mốc 01/2025 trong khi công ty bắt đầu 02/2025.
- Một sự cố production thật: triệu chứng, vai trò của bạn, bằng chứng nguyên nhân, hành động và kết quả.
- Một lần thay đổi yêu cầu hoặc bất đồng kỹ thuật có thật.
- Phạm vi công tác bạn sẵn sàng cam kết.
- Kinh nghiệm MES/ERP/thiết bị hoặc pipeline nào ngoài CV, nếu có.

Các thông tin này không cản trở ôn 103 câu. Chúng là phần còn thiếu để biến một số câu trả lời hành vi từ khung luyện tập thành câu chuyện cá nhân hoàn chỉnh.

Tài liệu được biên soạn và cá nhân hóa theo CV ngày 15/10/2026. Đã đọc đầy đủ bốn trang CV và đối chiếu một số điểm kiến trúc với mã nguồn TYT; chưa xác minh độc lập thành tích cá nhân hoặc các tình huống production. Các nguồn kỹ thuật được dẫn tại phần liên quan; API và cấu hình cần khớp phiên bản dự án.